

Contents

0.1	Instantiation Time	4
0.2	Runtime Representation	4
0.3	Reflection and Runtime Type Information	4
0.4	Type Safety	5
0.5	Performance	5
0.6	Summary	5
1	C++ Templates	6
1.1	Instantiation Time	6
1.1.1	Template Definition	6
1.1.2	Instantiation Trigger	6
1.1.3	Template Argument Substitution	7
1.1.4	Semantic Analysis of the Specialization	7
1.1.5	Intermediate Representation and Code Generation	7
1.1.6	Completion of the Instantiation Process	7
1.2	Runtime Representation	7
1.2.1	Runtime Form of Template Specializations	8
1.2.2	Monomorphization	8
1.2.3	Name Mangling	8
1.2.4	Final Executable Representation	8
1.2.5	Absence of Generic Metadata	9
1.3	Performance Characteristics	9
1.3.1	Static Dispatch and Elimination of Runtime Overhead	9
1.3.2	Compiler Optimization Opportunities	9
1.3.3	Zero-Overhead Abstraction and Trade-offs	9
1.4	Type Safety	10
1.4.1	Verification of Template Correctness	10
1.4.2	Example: Type Checking Across Specializations	10
1.4.3	Type Safety Across Instantiations	11
1.5	Reflection and Runtime Type Information	11
1.5.1	The <code>typeid</code> Operator and <code>std::type_info</code>	12
1.5.2	<code>dynamic_cast</code> and Polymorphic Template Classes	12
1.5.3	Limits of RTTI as a Reflection Mechanism	13
2	Java Generics	14
2.1	Instantiation Time	14
2.1.1	The Java Generic Instantiation Model	14
2.1.2	Type Erasure Pipeline	14
2.1.3	Instantiation with Type Bounds	15
2.1.4	Compiler Adaptation After Erasure	15
2.2	Runtime Representation	16
2.2.1	Erasure of Generic Type Parameters	16
2.2.2	Object Layout and Storage Representation	17

2.2.3	Inserted Runtime Casts	17
2.2.4	Bridge Methods	18
2.2.5	Primitive Types and Generic Representation	18
2.2.6	Comparison with C++ Templates	19
2.3	Performance Characteristics	19
2.3.1	Memory Representation and Pointer Indirection	20
2.3.2	Boxing and Unboxing Overhead	21
2.3.3	Runtime Effects of Type Erasure: Casts and Bridge Methods	21
2.3.4	Cache Locality and Data Layout	22
2.3.5	JVM Just-In-Time Optimization Techniques	22
2.3.6	Comparison with C++ Templates	23
2.3.7	Comparative Performance Summary	23
2.4	Type Safety	23
2.4.1	Static Type Checking in Java Generics	24
2.4.2	Bounded Type Parameters and Generic Constraints	25
2.4.3	Type Erasure and Preservation of Static Type Safety	25
2.4.4	Reification-Based Type Safety in C++ Templates	26
2.4.5	Comparison of Java Generics and C++ Templates	27
2.5	Reflection and Runtime Type Information	27
2.5.1	Reflection and RTTI Limitations	28
2.5.2	Type Tokens as an Explicit Runtime Representation	28
2.5.3	Comparison with C++ Template RTTI	29
3	C# Generics	31
3.1	Instantiation Time	31
3.1.1	CLR and JIT Generic Instantiation Mechanism	31
3.1.2	Comparison with C++ Templates: Runtime Instantiation vs Compile-Time Instantiation	32
3.1.3	Comparison with Java Generics: Runtime Instantiation vs Type Erasure . .	33
3.2	Runtime Representation	33
3.2.1	CLR Execution Engine and CIL Metadata Representation	34
3.2.2	Runtime Instantiation and JIT Compilation	34
3.2.3	Value-Type Specialization	35
3.2.4	Reference-Type Code Sharing	35
3.2.5	Runtime Dictionaries and Method Tables	36
3.2.6	Runtime Type Identity	36
3.2.7	Architectural Comparison	36
3.3	Performance Characteristics	37
3.3.1	Value-Type Instantiations: Memory Layout and Cache Efficiency	37
3.3.2	Reference-Type Instantiations: Indirection and Shared Code	38
3.3.3	Comparison with C++ Template Monomorphization	39
3.3.4	Comparison with Java Type-Erased Generics	40
3.3.5	Summary of Runtime Performance Characteristics	40
3.4	Type Safety	40
3.4.1	Static Type Checking of Generic Parameters	41
3.4.2	Generic Constraints as Type System Rules	42
3.4.3	Variance and Type-Safe Generic Subtyping	42
3.4.4	Reified Generics and Preservation of Type Information	43
3.4.5	Comparison with C++ Templates from a Type System Perspective	43
3.4.6	Type Safety Guarantees of the C# Generic Type System	44
3.5	Reflection and Runtime Type Information	44
3.5.1	Runtime Inspection of Generic Types Using System.Reflection	45
3.5.2	Comparison with C++ Templates and Java Generics	46

3.5.3	Comparative Summary of Runtime Generic Reflection	47
4	MetaProgramming	48
4.1	Overview of Metaprogramming	48
4.2	Compile-Time Computation Mechanisms in C++	48
4.2.1	Recursive Template Metaprogramming: Compile-Time Factorial	49
4.2.2	Recursive Template Metaprogramming: Compile-Time Fibonacci	50
4.2.3	Compile-Time Evaluation Using <code>constexpr</code> Functions	51
4.2.4	Compiler-Level Execution Model	51
4.2.5	Compile-Time Overload Resolution with SFINAE and C++20 Concepts	52
4.3	Compile-Time Overload Resolution with SFINAE and C++20 Concepts	54
4.4	Variadic C++ Template and other Metaprogramming Techniques	55
4.4.1	Template Template Parameters	56
4.4.2	Curiously Recurring Template Pattern (CRTP)	57
4.5	Limits of Java Generics in Replicating C++ SFINAE and Type Traits	58
4.6	Limits of C# Generics in Replicating C++ SFINAE and Type Traits	59
4.7	Instantiation-Time Checking and Metaprogramming Expressiveness	60
4.8	External Metaprogramming Workarounds in Java and C#	61
4.8.1	Java Reflection, Annotation Processing, and Code Generation	61
4.8.2	C# Source Generators and T4 Templates	62
4.9	Summary Practical Limits of Metaprogramming in C++, Java, and C# Generics	62

Parametric polymorphism provides a language-independent abstraction, its implementation varies considerably among programming languages and compiler architectures.

The principal design choice concerns how generic types are represented after compilation and when concrete type instantiations are produced.

These implementation strategies influence several aspects of the execution model, including runtime representation, reflection capabilities, performance, and binary size.

The following dimensions constitute the primary axes along which implementations of parametric polymorphism are typically compared.

0.1 Instantiation Time

A first distinguishing characteristic is the stage at which generic definitions are instantiated.

Some implementations perform instantiation entirely during compilation.

In this approach, each concrete use of a generic function or data structure is expanded into a specialized implementation before executable code is generated.

Since all specializations already exist in the final binary, runtime execution incurs no additional generic instantiation cost.

Other implementations compile generic definitions only once.

Generic type parameters are verified during type checking and subsequently removed or replaced by a common representation.

Consequently, no additional executable code is generated for different type arguments, and all instantiations share the same compiled implementation.

0.2 Runtime Representation

Implementations also differ in how generic types are represented after compilation.

Type erasure removes generic type parameters, producing a homogeneous representation where different instantiations share the same executable code.

The runtime system therefore treats different generic instantiations as equivalent representations.

Conversely, *reified generics* preserve concrete type information by generating distinct specialized code for each instantiated type.

Each generic instantiation may therefore possess its own runtime representation, memory layout, and executable instructions.

0.3 Reflection and Runtime Type Information

The availability of generic type information at runtime depends directly on the chosen representation strategy.

When type erasure is employed, generic parameters are generally unavailable after compilation.

Reflection mechanisms therefore provide limited access to instantiated generic types because most type information has been discarded.

Conversely, implementations based on reified generics preserve concrete type information throughout compilation.

Runtime reflection or runtime type identification (RTTI) may therefore recover the actual instantiated types, enabling operations such as generic type inspection, specialized serialization, and runtime metadata analysis.

0.4 Type Safety

Both implementation strategies preserve static type safety, although they achieve this objective differently.

In implementations based on type erasure, generic correctness is verified entirely during compilation.

Although type parameters disappear from the generated executable, compiler-inserted casts preserve the guarantees established by the static type system.

Implementations based on reified generics retain complete type information throughout compilation and generate type-specific implementations directly.

Since each specialization corresponds to a concrete type, correctness is preserved without requiring erased representations or implicit runtime casts.

0.5 Performance

The implementation strategy significantly influences both compilation and execution performance. Homogeneous implementations generally produce smaller binaries and reduce compilation time because generic definitions are compiled only once.

However, the shared representation may require additional indirections, boxing, or runtime casts, limiting optimization opportunities.

Heterogeneous implementations incur greater compilation cost and larger executables because every instantiated type generates a distinct implementation.

Nevertheless, the compiler can aggressively optimize each specialization, exploiting concrete type information for improved instruction selection, memory layout, inlining, and register allocation.

Consequently, runtime performance often approaches that of manually specialized code.

0.6 Summary

The implementation of parametric polymorphism is primarily characterized by five orthogonal dimensions:

1. the stage at which generic instantiation occurs;
2. the runtime representation of generic types;
3. the availability of runtime type information and reflection;
4. the trade-off between compilation cost, binary size, and execution performance;
5. the mechanisms through which static type safety is preserved.

These dimensions form the basis for comparing homogeneous implementations based on type erasure with heterogeneous implementations based on reification or monomorphization.

Chapter 1

C++ Templates

1.1 Instantiation Time

C++ implements parametric polymorphism through the *template* mechanism, which adopts a purely compile-time instantiation model. A template is not itself an executable entity but a parameterized blueprint from which the compiler generates independent concrete implementations. Executable code is generated only after the complete set of template arguments is known, so all instantiation is resolved during static compilation and no instantiation mechanism exists at runtime.

1.1.1 Template Definition

When the compiler encounters a template definition, it performs the front-end stages (lexical analysis, parsing, and the semantic checks possible without knowledge of future arguments) and stores a parameterized internal representation, without generating executable code:

```
template<typename T>
T get_max(T a, T b)
{
    return (a > b) ? a : b;
}
```

Since the compiler still ignores the concrete meaning of `T`, it cannot yet determine the operations, layout, or instructions required by the final implementation; the definition remains a blueprint awaiting instantiation.

1.1.2 Instantiation Trigger

Instantiation begins when the compiler encounters a concrete use of the template, either via explicit argument specification or implicit type deduction:

Example.

```
int i = get_max(3, 7);
double d = get_max<double>(2.5, 4.8);
```

The first invocation relies on deduction ($T = \text{int}$); the second specifies the argument explicitly ($T = \text{double}$). The compiler generates two distinct instantiations: `get_max<int>` and `get_max<double>`.

Each unique set of arguments constitutes a distinct instantiation, triggered through *implicit instantiation* (performed automatically when required), *explicit instantiation* (requested directly by the programmer), or *explicit specialization* (a wholly different implementation provided for a specific argument combination). In every case, code generation begins only once the complete set of arguments is known.

1.1.3 Template Argument Substitution

Once triggered, the compiler substitutes every formal parameter with its concrete argument:

From previous example. $T \rightarrow \text{int}$ produces:

```
int get_max(int a, int b)
{
    return (a > b) ? a : b;
}
```

while $T \rightarrow \text{double}$ produces the analogous specialization for `double`.

From this point, the compiler treats each specialization as if it had been written explicitly, behaving as an ordinary non-template function.

1.1.4 Semantic Analysis of the Specialization

Instantiation does not immediately produce executable code: each generated specialization first undergoes the normal semantic analysis applied to ordinary C++ entities, including type checking, overload resolution, name lookup, constraint verification, constant expression evaluation, and instantiation of nested templates. This step is essential because many semantic properties cannot be verified before substitution occurs; consequently, a syntactically correct template may still fail to compile if a specific instantiation violates the requirements imposed by the template body.

1.1.5 Intermediate Representation and Code Generation

Once semantic analysis succeeds, the specialization enters the compiler back-end, which generates an Intermediate Representation, applies the standard optimization pipeline (constant propagation, dead-code elimination, inlining, loop optimizations, register allocation, instruction selection), and emits native machine code exclusively for that specialization. This process repeats independently for every distinct instantiation, so each specialization owns an independent executable implementation; no shared representation is produced that could serve multiple template arguments.

1.1.6 Completion of the Instantiation Process

Instantiation terminates entirely before linking: by the time object files are generated, every specialization required by the translation unit already exists as compiled code, and the final linked executable contains all of them. The lifecycle can be summarized as: parsing the definition, storing it as a parameterized representation, triggering instantiation on concrete use, substituting arguments, performing semantic analysis, generating IR, optimizing, and emitting native code included in the executable before execution begins. The defining characteristic of C++ parametric polymorphism is therefore that the entire process completes during static compilation; when execution begins, no instantiation mechanism remains active, and the runtime simply executes machine code already generated for every required specialization.

1.2 Runtime Representation

C++ templates transform generic source-level abstractions into concrete program entities before execution. A template declaration does not itself correspond to a runtime object; it defines a family of possible entities materialized only when the compiler generates a specific specialization. The runtime representation of a specialization is therefore identical to that of an ordinary function, class, or object: after linking, the executable contains native instructions and data associated with each instantiated specialization, and the loader and processor have no knowledge of the original template or its parameters. Conceptually, a template is a compile-time transformation $T : (A_1, \dots, A_n) \rightarrow P$, so that $T\langle \text{int} \rangle \rightarrow P_{\text{int}}$ and $T\langle \text{double} \rangle \rightarrow P_{\text{double}}$ are independent implementations; at runtime the program never invokes a generic representation of T , only the machine code generated for each concrete specialization.

1.2.1 Runtime Form of Template Specializations

Consider the following function template:

```
template <typename T>
T add(T a, T b)
{
    return a + b;
}
```

When the program requires `add<int>(1,2)` and `add<double>(1.0,2.0)`, the compiler produces two separate functions, `add<int>(int,int)` and `add<double>(double,double)`, represented at the binary level as independent machine-code regions with their own instructions, symbol identity, and entry address. The runtime system contains no generic function body carrying a runtime type parameter, only the concrete implementations produced during translation; control flow transfers directly to the generated addresses, and the processor executes without any awareness that the code originated from a template. Hence the runtime representation of `Vector<int>` and `Vector<double>` is not a single generic object `Vector<T>`, but two independent concrete types produced before execution.

1.2.2 Monomorphization

C++ uses heterogeneous translation, commonly called *monomorphization*: the compiler substitutes template arguments into the definition and generates a concrete function or class implementation for each argument set used by the program. For example:

```
template <typename T>
struct Container
{
    T value;
};

Container<int> a;
Container<float> b;
```

creates two independent specializations, `Container<int>` and `Container<float>`; the executable contains their concrete representations, with no runtime object representing `Container<T>` since the generic abstraction was already resolved during translation. For function templates, each specialization produces a distinct machine-code artifact behaving exactly like a function written explicitly for that type, with its own entry point invoked through normal control-flow instructions.

1.2.3 Name Mangling

Compilers encode template arguments into symbol names through *name mangling*, allowing different specializations to coexist in object files and letting the linker distinguish entities such as `function<int>()` and `function<double>()`. This is purely a compiler/linker mechanism, not runtime metadata: once linking completes, the CPU does not interpret mangled names, and an executable address identifies machine code without any awareness of the source-level template relationship.

1.2.4 Final Executable Representation

After linking, the executable is a collection of concrete compiled entities, $Executable = \{Code(P_1), \dots, Code(P_n)\}$, where each P_i corresponds to a template specialization. It contains machine instructions for instantiated functions, symbols identifying them, relocation information, and optionally runtime metadata produced by other C++ mechanisms (Section 1.5) – artifacts no different from those produced by non-template code. During execution the processor never interprets template parameters: a call instruction transfers control to a compiled function’s address, and the link back to the original template source survives only in debugging information or symbol naming conventions, not in the runtime execution model.

1.2.5 Absence of Generic Metadata

A defining characteristic of the C++ model is the absence of runtime generic metadata: after instantiation, the executable retains no information describing the original template declaration or its possible arguments. Given `template<typename T> void process(T value);`, the runtime environment only observes concrete functions such as `process<int>()` or `process<double>()` – template declarations, parameter lists, generic type variables, and mechanisms for creating new specializations dynamically simply do not exist as runtime entities; the execution environment manipulates only the final native representation. The limited runtime type identification C++ does provide is discussed in Section 1.5.

1.3 Performance Characteristics

The performance characteristics of C++ templates derive directly from monomorphization (Section 1.2.2): every instantiation is transformed into an independent specialization operating on a concrete type, so the executable contains ordinary statically typed functions and classes tailored to each instantiated type rather than a universal generic implementation. For example:

```
template<typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}
```

instantiated with different types generates specialized functions equivalent to handwritten `int max(int,int)` and `double max(double,double)` versions, whose machine code operates directly on concrete data types without runtime mechanisms for resolving the generic parameter.

1.3.1 Static Dispatch and Elimination of Runtime Overhead

One of the main performance advantages of monomorphization is static dispatch: since the concrete type of every parameter is known at compile time, function calls are resolved before execution, eliminating the indirections required by dynamic dispatch. Given `template<typename T> void process(T& object) { object.compute(); }` called with a `Matrix`, the compiler resolves the call directly to `Matrix::compute()`, reducing overhead and exposing further optimization opportunities. A related consequence is the elimination of boxing and unboxing: template-generated code operates directly on the native representation of values, so `std::vector<int>` stores integers in their native form without conversion into a generic object wrapper.

1.3.2 Compiler Optimization Opportunities

Because every specialization is fully typed, optimizing compilers apply the same techniques used for manually written specialized code. A fundamental example is function inlining: since both implementation and argument types are known at compile time, `template<typename T> inline T square(T x) { return x*x; }` called as `square(5)` can be replaced directly by `5*5`, removing call overhead and enabling further constant propagation, constant folding, dead-code elimination, loop optimization, and strength reduction. Template parameters can also encode compile-time constants – e.g. a `template<int N>` array bound lets the compiler know the exact iteration count of a loop and apply loop unrolling. Compile-time specialization also enables branch elimination via `if constexpr`, where only the branch matching the selected type is emitted, and improves automatic vectorization: because concrete types and memory layouts are known, numerical templates such as an element-wise array `add` can be compiled directly into SIMD instructions.

1.3.3 Zero-Overhead Abstraction and Trade-offs

The combination of static dispatch, direct data representation, and compiler optimization enables C++ templates to follow the *zero-overhead abstraction* principle. According to this principle, abstractions introduced at the source level should not impose additional runtime costs compared with an equivalent manually specialized implementation. Template-based abstractions can therefore provide generic programming capabilities while maintaining execution efficiency comparable

to explicitly written type-specific code.

The main performance limitation associated with template monomorphization is related to the expansion of generated code. Since every specialization produces an independent implementation, extensive template usage may increase executable size. A larger instruction footprint can negatively affect instruction cache locality, increasing instruction cache misses in performance-critical applications.

This effect does not originate from additional runtime computation introduced by templates, but from the interaction between generated specialized code and the processor memory hierarchy. Therefore, template performance is generally dominated by the benefits of compile-time specialization, while instruction cache behavior represents the main architectural factor capable of reducing these advantages.

1.4 Type Safety

Static type safety represents one of the fundamental objectives of parametric polymorphism. Its purpose is to guarantee that every operation performed on a value is compatible with its statically known type before program execution. In this way, incorrect type interactions are detected during compilation rather than being deferred to runtime.

Different implementations of parametric polymorphism achieve static type safety through different mechanisms, as introduced in the overview of implementation strategies. C++ templates adopt a reified implementation strategy based on the template instantiation and monomorphization process described in Section 1.2.2: type parameters are never erased, and every instantiation is checked against a concrete, fully specialized implementation.

From a type system perspective, reification allows the compiler to retain complete information about the actual types involved in every specialization. Unlike erased representations, where the original type parameter is no longer available during execution, C++ templates preserve the relationship between the generic definition and the concrete types used to instantiate it. Consequently, all type-dependent operations are checked directly against the substituted types.

1.4.1 Verification of Template Correctness

The verification of template correctness occurs through the ordinary C++ static type system. After template argument substitution, the compiler performs semantic analysis on the generated specialization, checking that all expressions, operators, conversions, and function calls are valid for the concrete type. If a required operation is not supported by the instantiated type, the specialization is considered ill-formed and the program is rejected during compilation.

This mechanism differs fundamentally from implementations based on type erasure because C++ templates never require the compiler to recover discarded type information. Since each specialization is generated from a known concrete type, operations are resolved statically according to the rules of the language type system. Therefore, no implicit runtime casts are required to restore the original generic type information.

1.4.2 Example: Type Checking Across Specializations

The following example illustrates how C++ templates preserve static type safety through concrete specialization.

```
#include <string>

template <typename T>
T maximum(const T& a, const T& b)
{
    return (a < b) ? b : a;
}

int main()
{
```

```

int x = maximum(3, 7);

double y = maximum(3.5, 8.1);

// Compilation error:
// no valid specialization exists because
// the arguments do not have the same type.

// auto z = maximum(3, std::string("hello"));
}

```

The function template `maximum` describes a generic operation that can be applied to any type satisfying the required semantic constraints. When the compiler encounters the expression `maximum(3, 7)`, template argument deduction determines that the parameter `T` corresponds to the type `int`. The compiler then creates a specialization equivalent to:

```
int maximum(const int& a, const int& b);
```

Similarly, the invocation `maximum(3.5, 8.1)` produces a specialization where `T = double`, and the generated function operates exclusively on values of type `double`.

Each specialization is independently checked by the compiler. The comparison operator `<` must exist for the instantiated type, the return expression must be compatible with the declared return type, and all parameter bindings must satisfy the requirements imposed by the C++ type system.

The invalid invocation `maximum(3, std::string("hello"))` cannot produce a valid specialization because template argument deduction cannot identify a single type for `T`. The compiler therefore rejects the program before executable code is generated. No runtime type inspection, dynamic cast, or conversion mechanism is involved, because the type inconsistency is detected entirely during static analysis.

1.4.3 Type Safety Across Instantiations

A further consequence of template reification is that type checking is performed with respect to each concrete template instantiation. A template definition represents a family of possible programs, while every generated specialization represents a fully specified program whose types are known. Therefore, correctness is established separately for every instantiated version rather than through a single erased generic representation.

This property means that template abstraction does not weaken the C++ static type system. Instead, templates delay the determination of concrete types until instantiation, after which normal compile-time type checking is applied to the resulting program. Every successfully generated specialization is consequently an ordinary statically typed C++ entity, whose correctness has been verified before execution.

From a type-theoretic perspective, C++ templates achieve static type safety through reification: generic definitions preserve type information, template arguments are substituted with concrete types, and the resulting specializations are validated according to the complete rules of the language type system. This approach guarantees generic correctness without requiring erased representations or compiler-generated runtime casts.

1.5 Reflection and Runtime Type Information

As introduced in the overview of implementation strategies, the availability of generic type information at runtime depends directly on whether the implementation follows a type-erasure or a reified model. C++ templates follow the reified model described in Section 1.2: template parameters are not represented as a single runtime generic entity, and monomorphization preserves the identity of the concrete types used during compilation. However, compile-time type preservation should not be confused with runtime reflection. After compilation, the C++ execution environment does not generally provide a universal mechanism capable of enumerating arbitrary

template parameters, inspecting source-level declarations, or dynamically querying all properties of instantiated template entities. What C++ does provide is a restricted mechanism, Run-Time Type Information (RTTI), which allows limited runtime identification of concrete types.

1.5.1 The typeid Operator and std::type_info

The primary standard facilities for RTTI are the operators `typeid` and `dynamic_cast`, together with the `std::type_info` class.

The `typeid` operator obtains runtime type information associated with an expression or a type. When applied to an instantiated template class, it can identify the concrete specialization generated by the compiler. For example:

```
template <typename T>
class Wrapper {
};

Wrapper<int> a;
Wrapper<double> b;

std::cout << typeid(a).name();
std::cout << typeid(b).name();
```

The two objects correspond to different C++ types, namely `Wrapper<int>` and `Wrapper<double>`. Consequently, the RTTI system treats them as distinct runtime types whenever RTTI metadata is generated for these entities. The resulting `std::type_info` objects represent the runtime identity of the instantiated classes.

The information exposed by `std::type_info` is intentionally limited. It allows comparison of runtime type identities through operations such as equality comparison and provides an implementation-defined textual representation through `name()`. It does not provide access to template arguments, class members, inheritance hierarchies, or source-level metadata in the manner of a full reflection system.

1.5.2 dynamic_cast and Polymorphic Template Classes

The `dynamic_cast` operator provides another RTTI mechanism for performing safe runtime conversions within polymorphic class hierarchies. A class must contain at least one virtual function for RTTI-based dynamic identification to be available.

Template classes can participate in such hierarchies like ordinary C++ classes. For example:

```
class Base {
public:
    virtual ~Base() = default;
};

template <typename T>
class Derived : public Base {
};

Base* ptr = new Derived<int>();

Derived<int>* result = dynamic_cast<Derived<int>*>(ptr);
```

During execution, `dynamic_cast` uses runtime type information associated with the object to verify whether the actual object type corresponds to the requested target type. In this example, the cast succeeds because the dynamically stored object is an instance of the concrete template specialization `Derived<int>`.

The same operation would fail for an object instantiated with a different template argument,

such as `Derived<double>`, because each template specialization represents a distinct C++ type. Therefore, RTTI preserves the identity of instantiated template classes when those classes are part of a polymorphic hierarchy.

1.5.3 Limits of RTTI as a Reflection Mechanism

C++ templates demonstrate a hybrid relationship between type preservation and runtime identification. The template mechanism preserves concrete type information during compilation through specialization generation, but this information is not automatically exposed through a general runtime reflection interface.

The compiler knows the complete instantiated type during template compilation. It can therefore perform static analysis, select specialized implementations, and generate code based on the concrete template arguments. This information belongs to the compile-time type system and is consumed before program execution.

RTTI, in contrast, represents a restricted runtime mechanism. It operates only on the subset of type information explicitly retained by the C++ object model, primarily for polymorphic objects and explicit type identification through `typeid`. For example, a template class such as:

```
template<typename T>
class Base
{
public:
    virtual ~Base(){}
};
```

```
Base<int> object;
```

may generate runtime type information describing `Base<int>`, but it does not generate a runtime descriptor describing the generic definition `Base<T>`, because the generic template abstraction has no runtime existence (Section 1.2).

Consequently, RTTI can distinguish between instantiated template types when the compiler emits the corresponding runtime metadata, but it cannot reconstruct the complete template definition or inspect arbitrary template parameters dynamically. C++ templates therefore preserve generic type information through compile-time monomorphization rather than through runtime reification, and standard RTTI mechanisms provide runtime access to the identity of concrete instantiated types without constituting a complete reflection system.

Chapter 2

Java Generics

2.1 Instantiation Time

Java Generics adopt a compilation strategy based on compile-time instantiation followed by type erasure. During compilation, the Java compiler associates concrete type arguments with generic declarations in order to perform static type checking. However, this association does not lead to the generation of a new specialized class representation.

This design differs fundamentally from the approach adopted by C++ templates. While Java uses instantiation only as a compile-time type verification mechanism, C++ performs template instantiation through monomorphization, generating a separate specialization for every concrete type argument.

2.1.1 The Java Generic Instantiation Model

Consider the following generic declaration:

```
class Box<T> {  
    private T value;  
  
    public T get() {  
        return value;  
    }  
}
```

When the compiler encounters:

```
Box<String> box = new Box<String>();
```

the type parameter T is instantiated with the argument `String` for the purpose of compile-time verification. The compiler therefore checks that every operation involving `box` respects the constraint:

$$T = \text{String}$$

However, this instantiation does not generate a new runtime class:

$$\text{Box} < \text{String} >$$

Instead, Java produces a single generic class representation that is shared by all possible instantiations.

2.1.2 Type Erasure Pipeline

After generic type checking, the Java compiler applies type erasure. The type parameter is removed and replaced by its erasure according to the rules defined by the Java Language Specification.

For an unbounded parameter:

$$< T >$$

the erased type becomes:

$$Object$$

Therefore:

```
class Box<T>
```

is transformed into a representation equivalent to:

```
class Box {  
    Object value;  
  
    Object get()  
}
```

At this point, Java has completed the generic instantiation process. No additional class corresponding to:

$$Box < String >$$

exists in the JVM.

2.1.3 Instantiation with Type Bounds

When a Java generic parameter has a bound:

```
class NumericBox<T extends Number>
```

the compiler verifies that every instantiation satisfies:

$$T <: Number$$

The bound also determines the erasure type. According to the Java Language Specification, the erasure of a type variable corresponds to the erasure of its leftmost bound.

Therefore:

```
NumericBox<Integer>  
NumericBox<Double>
```

are both translated into a representation equivalent to:

```
NumericBox
```

with:

```
Number value;
```

2.1.4 Compiler Adaptation After Erasure

Because Java removes generic parameters before execution, the compiler must insert additional bytecode instructions to preserve the semantics established during compile-time instantiation.

For example:

```
String s = box.get();
```

is compiled into a call returning:

```
Object get()
```

followed by an inserted cast:

```
checkcast java/lang/String
```

Across all these mechanisms, the Java instantiation model consistently contrasts with C++ template compilation: whereas C++ monomorphization generates a distinct specialization for every concrete type argument, with type information encoded directly into each generated implementation, Java instantiation is entirely a compile-time verification step that resolves to a single shared runtime representation, requiring the compiler to insert casts and rely on erasure rules precisely because no specialized, type-specific code is produced.

2.2 Runtime Representation

Java Generics are implemented through a *type erasure* model, in which generic type parameters are removed from the executable representation before runtime. The generic abstraction exists only during compilation, where the compiler performs static type checking, verifies type constraints, and generates bytecode compatible with the non-generic JVM execution model.

The fundamental design principle of Java Generics is that different parameterizations of the same generic class share a single runtime representation. For example, the source-level types

$$List < String > \quad \text{and} \quad List < Integer >$$

do not correspond to two different classes inside the JVM. Instead, both are represented by the same runtime class:

List

The JVM runtime does not maintain separate metadata, object layouts, or method implementations for different generic arguments.

2.2.1 Erasure of Generic Type Parameters

The Java compiler transforms parameterized types by replacing type variables with their erasures. According to the Java Language Specification, the erasure of an unbounded type variable is `Object`, while a bounded type variable is replaced by the erasure of its first bound.

Consider the generic declaration:

```
class Box<T> {
    private T value;

    T get() {
        return value;
    }
}
```

After erasure, the runtime representation is equivalent to:

```
class Box {
    private Object value;

    Object get() {
        return value;
    }
}
```

The JVM therefore executes a representation where the type variable *T* has been replaced by its erased type. The runtime system has no knowledge that the original source-level declaration used

a generic parameter.

Formally, the transformation can be expressed as:

$$Box < T > \rightarrow Box < Object >$$

for an unbounded type variable.

Consequently, the runtime identity of an object is determined only by the erased class:

$$RuntimeType(Box < String >) = RuntimeType(Box < Integer >) = Box$$

The generic parameter is not part of the runtime type identity.

2.2.2 Object Layout and Storage Representation

Because Java Generics do not create specialized runtime classes, generic objects use the same memory representation as ordinary JVM objects.

A generic reference field:

T value

is represented after erasure as:

Object value

The JVM stores a reference to an object rather than a value specialized for the concrete generic type.

For example, an instance of:

ArrayList < String >

and an instance of:

ArrayList < Integer >

have identical internal structural representations. The JVM does not allocate a specialized array of strings or integers for these collections. The generic container stores references using the standard JVM reference representation.

This differs from a reified implementation, such as C++ template monomorphization, where each concrete instantiation may generate a distinct object layout and specialized machine code. In Java, the same erased representation is reused for all parameterizations.

2.2.3 Inserted Runtime Casts

Since generic parameters are removed before execution, the compiler inserts runtime casts whenever a value retrieved from a generic structure must be treated as a more specific type.

Example:

```
String s = list.get(0);
```

The erased execution model becomes equivalent to:

```
Object tmp = list.get(0);
String s = (String) tmp;
```

At the bytecode level, the JVM executes a **checkcast** instruction.

The runtime sequence is therefore:

retrieve erased reference → check actual object type → use typed reference

The runtime verification concerns the concrete class of the object, not the generic argument. The JVM checks whether the object is a **String**, but it does not verify the existence of an original **List<String>** instantiation.

2.2.4 Bridge Methods

Type erasure can modify method signatures and break overriding relationships at the JVM level. To preserve polymorphism, the Java compiler generates synthetic *bridge methods*.

Example:

```
class Node<T> {
    T get();
}

class StringNode extends Node<String> {
    String get();
}
```

Before erasure, the overriding relationship is:

Node < T > .get() : T

overridden by:

StringNode.get() : String

After erasure, the parent method becomes:

get() : Object

while the child method remains:

get() : String

Since the JVM method descriptors differ, the compiler generates a bridge:

```
Object get() {
    return get();
}
```

The bridge method adapts the specialized method to the erased representation, allowing normal JVM method dispatch to operate correctly.

Bridge methods are therefore a direct runtime consequence of the decision to implement generics through erasure rather than through specialized generated types.

2.2.5 Primitive Types and Generic Representation

Java Generics are restricted to reference types because erased generic parameters are represented using reference-compatible types.

A primitive instantiation:

List < int >

is not permitted.

Instead, the corresponding wrapper type must be used:

$$List < Integer >$$

The runtime representation therefore becomes:

$$int \rightarrow Integer \rightarrow Object$$

The primitive value is converted into an object representation through boxing, and retrieved values require unboxing.

This behavior follows directly from the erased representation: since every generic parameter is mapped to a reference type, the JVM cannot create a specialized generic representation containing primitive values.

2.2.6 Comparison with C++ Templates

C++ Templates are based on a different runtime representation strategy: *monomorphization*. Unlike Java Generics, C++ compilers generate a separate representation for each concrete template instantiation.

This difference explains why Java uses one erased runtime representation:

$$List < T > \rightarrow List < Object >$$

while C++ can generate independent specialized representations:

$$Vector < int >, Vector < double >, \dots$$

The Java approach avoids multiple generated implementations but sacrifices runtime visibility of generic parameters and requires mechanisms such as casts, boxing, and bridge methods. C++ preserves concrete type information in the generated executable by creating specialized versions, but this may increase generated code size.

The central distinction is therefore:

Java Generics preserve abstraction through erasure

whereas:

C++ Templates preserve abstraction through specialization

For Java, the generic type parameter is a compile-time concept. For C++, template instantiation becomes part of the generated runtime representation.

2.3 Performance Characteristics

The execution performance of generic programming is strongly determined by the translation strategy adopted by the language implementation. Java Generics and C++ templates represent two fundamentally different approaches: Java employs a *type erasure* model, whereas C++ relies on compile-time instantiation and *monomorphization*. These strategies produce different runtime representations and consequently different execution characteristics.

Java Generics are implemented through type erasure, as specified by the Java Language Specification. During compilation, generic type parameters are removed from the generated bytecode and replaced by their upper bounds, typically `Object`. Generic information is preserved only in

metadata attributes used by the compiler and runtime tools, but it does not determine the actual machine representation of executed objects.

For example, the generic class:

```
class Container<T> {  
    T value;  
  
    T get() {  
        return value;  
    }  
}
```

is translated approximately into:

```
class Container {  
    Object value;  
  
    Object get() {  
        return value;  
    }  
}
```

Consequently, the JVM executes a single implementation shared by all generic instantiations. A `Container<String>` and a `Container<Integer>` therefore use the same erased runtime representation.

C++ templates follow a different execution model. Template instantiation occurs during compilation, and the compiler generates a specialized version of the code for every concrete type used by the program. For example:

```
template <typename T>  
class Container {  
  
    T value;  
  
public:  
  
    T get() {  
        return value;  
    }  
};
```

instantiated as:

```
Container<int> a;  
Container<double> b;
```

produces two independent implementations specialized for `int` and `double`. This process, known as monomorphization, allows the compiler to exploit complete type information during optimization.

Therefore, Java preserves a uniform runtime representation by removing generic type information, while C++ exchanges additional generated code for type-specific machine implementations.

2.3.1 Memory Representation and Pointer Indirection

The main performance consequence of Java type erasure is that generic structures operate primarily on object references rather than direct values. In the JVM object model, reference variables contain pointers to heap objects, requiring additional memory accesses compared with direct value storage.

For example:

```
ArrayList<Integer> values = new ArrayList<>();

values.add(10);

int x = values.get(0);
```

does not store the integer value directly inside the container. Instead, the runtime representation consists of a reference to an `Integer` object:

Array reference $\rightarrow$ Object reference $\rightarrow$ Heap object containing value

Accessing an element therefore requires additional pointer dereferencing and memory accesses.

In contrast, C++ templates combined with value semantics allow direct storage of objects inside contiguous memory regions. For example:

```
std::vector<int> values;
```

stores integer values directly:

$\text{int} \rightarrow \text{int} \rightarrow \text{int} \rightarrow \text{int}$

The absence of intermediate references reduces memory latency and improves the ability of the processor to exploit cache locality (a systematic comparison with C++ is given in Section 2.3.6).

2.3.2 Boxing and Unboxing Overhead

A significant limitation of Java Generics is the inability to directly instantiate generic types with primitive values. Generic parameters must always refer to object types; therefore primitive values require wrapper objects.

For example:

```
ArrayList<Integer> values = new ArrayList<>();

values.add(5);
```

requires automatic boxing:

$\text{int} \rightarrow \text{Integer}$

while retrieving the value requires unboxing:

$\text{Integer} \rightarrow \text{int}$

The boxing process introduces additional execution costs:

- creation of wrapper objects when values are not reused;
- additional memory accesses caused by object references;
- increased memory footprint due to object metadata;
- conversion operations during boxing and unboxing.

These overheads are particularly relevant for numerical workloads, where large collections of primitive values are processed repeatedly (C++ templates avoid this limitation entirely, as discussed in Section 2.3.6).

2.3.3 Runtime Effects of Type Erasure: Casts and Bridge Methods

Type erasure also requires Java to preserve type safety through inserted casts. Since the JVM executes erased bytecode, the compiler introduces runtime checks when retrieving values from

generic structures.

For example:

```
String value = list.get(0);
```

is internally equivalent to:

```
String value = (String) list.get(0);
```

The cast introduces an additional runtime verification step. Modern JVM implementations can frequently eliminate redundant checks through JIT optimization, but unpredictable execution paths may prevent complete removal.

Another consequence of type erasure is the generation of synthetic bridge methods. Consider:

```
interface Box<T> {  
    T get();  
}  
  
class StringBox implements Box<String> {  
    public String get() {  
        return "value";  
    }  
}
```

After erasure, the interface method becomes:

```
Object get();
```

Therefore, the compiler generates an additional bridge method:

```
public Object get() {  
    return get();  
}
```

The additional invocation layer can introduce a small runtime overhead. However, HotSpot can often remove this cost through method inlining when the runtime profile provides sufficient information.

2.3.4 Cache Locality and Data Layout

The memory layout produced by generic structures strongly influences cache behavior.

A Java container containing reference types generally follows the layout:

Container storage → Reference → Heap object → Value

Objects may be distributed across different regions of memory, reducing spatial locality. During sequential traversal, the processor may need to load multiple independent cache lines because consecutive logical elements are not necessarily physically adjacent.

This increases cache misses and reduces effective memory bandwidth, in contrast with the contiguous value storage achievable through C++ templates (Section 2.3.6).

2.3.5 JVM Just-In-Time Optimization Techniques

Although Java Generics introduce runtime overheads, modern JVM implementations attempt to reduce these costs through adaptive optimization.

The HotSpot JVM performs runtime profiling and dynamically generates optimized machine code for frequently executed paths. Relevant techniques include:

- **Devirtualization:** replacing dynamic method resolution with direct calls when the receiver type is predictable.
- **Inline caching:** recording observed runtime types at call sites to optimize repeated method invocations.
- **Escape analysis:** determining whether objects remain local to a method and enabling scalar replacement or elimination of unnecessary allocations.

For example, if an object created inside a method never escapes, the JVM may replace the object allocation with direct scalar values, reducing memory access overhead. However, these optimizations remain speculative and depend on runtime profiling, unlike the ahead-of-time specialization guaranteed by C++ templates.

2.3.6 Comparison with C++ Templates

The preceding subsections isolate individual runtime costs introduced by Java’s type-erasure model; this section consolidates them into a single systematic comparison with C++ templates.

At the representation level, Java containers store object references, introducing pointer indirection and additional memory accesses, whereas C++ templates instantiated with concrete types allow direct, contiguous value storage. This layout difference directly affects cache behavior: Java’s reference-based objects can be scattered across the heap, increasing cache misses during traversal, while C++ value layouts enable efficient cache-line utilization, hardware prefetching, and vectorization.

At the level of primitive handling, Java’s inability to instantiate generics with primitive types forces boxing and unboxing whenever numeric values are stored in generic structures, adding allocation and conversion costs absent from C++, where a template instantiated with a primitive type manipulates its native representation directly.

At the level of code generation, the two models also diverge on instruction cache behavior. Java type erasure generates a single implementation shared among all generic instantiations, keeping executable size small but preventing the generated code from exploiting concrete type information. C++ monomorphization instead generates a separate implementation per instantiated type: this increases executable size and can introduce instruction-cache pressure (“code bloat”), but enables stronger optimizations such as complete function inlining, constant propagation, and architecture-specific code generation.

Finally, the two models differ in *when* optimization decisions are made. C++ performs optimization during ahead-of-time compilation, with the compiler having complete information about data representation, memory layout, and available operations before the program ever runs. Java follows a dynamic model instead, in which the JVM profiles the running program and generates JIT-optimized machine code based on observed behavior. Consequently, C++ provides predictable, guaranteed specialization, while Java provides adaptive optimization capable of exploiting information that is only available at runtime.

2.3.7 Comparative Performance Summary

2.4 Type Safety

Type safety is a fundamental property of statically typed programming languages and defines the ability of a type system to prevent invalid operations on values before program execution. In the context of parametric polymorphism, type safety requires that abstract type parameters can be substituted with concrete types without introducing operations that violate the constraints established by the program.

A sound type system guarantees that every expression accepted by the compiler respects the typing rules of the language. This property is traditionally described through two fundamental

Characteristic	Java Generics	C++ Templates
Generic representation	Type erasure with shared run-time representation	Compile-time monomorphization
Primitive handling	Boxing and unboxing required	Direct native representation
Memory access	Reference indirection and pointer chasing	Direct contiguous storage
Dispatch	Potential runtime dispatch	Static compile-time dispatch
Optimization	Adaptive JIT optimization	AOT compiler optimization
Cache locality	Reduced by object-based layouts	Improved through value layouts

Table 2.1: Runtime performance comparison between Java Generics and C++ templates.

principles:

- *Progress*: a well-typed program cannot reach a state where an undefined operation caused by a type mismatch must be performed;
- *Preservation*: the evaluation of a well-typed expression preserves its type correctness.

Parametric polymorphism extends these guarantees by allowing the definition of algorithms and data structures independently from specific types. However, the implementation strategy adopted by a language determines how type information is preserved during compilation and how the type system guarantees correctness.

The implementation strategy adopted by Java and C++ differs in how type information is preserved after compilation; this distinction is examined in detail in Section 2.4.5, once the static type checking rules of Java Generics have been introduced.

2.4.1 Static Type Checking in Java Generics

Java Generics extend the Java type system by introducing parameterized types. A generic declaration defines a family of related types whose correctness depends on the constraints imposed on their type parameters.

Consider the following generic class:

```
class Box<T> {
    private T value;

    public void set(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }
}
```

The type parameter `T` represents an abstract type that is determined when the generic class is instantiated.

For example:

```
Box<String> textBox = new Box<>();
textBox.set("Java");
String value = textBox.get();
```

The compiler interprets `Box<String>` as a type where every use of `T` must be compatible with

String. Therefore, the following operation is rejected:

```
textBox.set(10);
```

because the argument type **Integer** does not satisfy the type constraint established by the parameterization.

The type system therefore prevents the creation of inconsistent states: after the declaration of **Box<String>**, every value inserted into the container must conform to the declared type argument.

The same principle applies to method invocations. If a generic method requires a specific type relationship, the compiler verifies that all actual arguments satisfy the required constraints before accepting the program.

For example:

```
static <T> T identity(T value) {  
    return value;  
}  
  
String result = identity("example");
```

The compiler infers the type argument and ensures that the inferred substitution preserves the validity of the expression.

2.4.2 Bounded Type Parameters and Generic Constraints

Java generics also support bounded type parameters, allowing the type system to restrict the set of admissible substitutions.

For example:

```
class Maximum<T extends Comparable<T>> {  
    T max(T a, T b) {  
        return a.compareTo(b) > 0 ? a : b;  
    }  
}
```

The declaration:

```
<T extends Comparable<T>>
```

introduces a constraint stating that every valid type argument must provide a comparison operation compatible with the generic algorithm.

The compiler verifies this constraint when the generic type is instantiated. A type that does not satisfy the required relationship is rejected statically.

Therefore, Java generic constraints express properties that must hold for every valid substitution of the type parameter, allowing the type system to reason about generic programs independently from concrete types.

2.4.3 Type Erasure and Preservation of Static Type Safety

Java implements generics through type erasure. From the perspective of the type system, this means that generic parameters are used during static analysis but are not represented as independent types after the compilation process.

The important property is that erasure does not remove the guarantees provided by the type system. Generic correctness is established before the erasure transformation occurs.

For example:

```
List<String> names = new ArrayList<>();  
  
names.add("Alice");  
  
String name = names.get(0);
```

The compiler verifies that:

- only values compatible with **String** can be inserted into the list;
- every retrieval operation is interpreted as producing a value of type **String**.

After the generic verification phase, the implementation no longer depends on the explicit parameter **String**. However, the soundness argument of the type system is based on the fact that all possible uses of the generic abstraction have already been checked.

Consequently, Java separates the concepts of:

- *static type correctness*, which is guaranteed by generic type checking;
- *type representation*, which is determined after the generic analysis phase.

The type-erasure strategy therefore preserves type safety by shifting the responsibility for correctness entirely to compile-time verification.

2.4.4 Reification-Based Type Safety in C++ Templates

C++ templates adopt a different approach based on type reification. Instead of removing type parameters after checking, the compiler instantiates templates using concrete type arguments.

Consider:

```
template <typename T>  
class Box {  
  
private:  
  
    T value;  
  
public:  
  
    void set(T value) {  
        this->value = value;  
    }  
  
    T get() {  
        return value;  
    }  
};
```

A concrete use:

```
Box<int> integerBox;  
  
integerBox.set(10);  
  
int value = integerBox.get();
```

causes the compiler to analyse the template using the concrete type **int**. Every operation involving **T** is therefore checked against the actual instantiated type.

If an invalid operation is attempted:

```
Box<int> box;  
  
box.set("text");
```

the compiler rejects the program because the instantiated type **int** does not accept a string value.

From the perspective of type safety, C++ templates preserve the complete type information during template instantiation. The compiler verifies the correctness of each concrete specialization directly rather than checking an abstract erased representation.

2.4.5 Comparison of Java Generics and C++ Templates

Java Generics and C++ Templates both provide compile-time guarantees through parametric polymorphism, but they preserve type information in different ways.

Java follows the model:

```
Generic abstraction
    |
    v
Static type checking
    |
    v
Type erasure
```

The type system verifies every substitution before removing the generic parameter from the resulting representation.

C++ follows the model:

```
Template abstraction
    |
    v
Instantiation with concrete type
    |
    v
Static type checking of specialization
```

The compiler retains the concrete type throughout template processing and verifies operations directly on the instantiated form.

Therefore, the fundamental distinction is not whether type safety is provided, but how the type system associates abstract parameters with concrete types. Java achieves safety through static verification before erasure, whereas C++ achieves safety through compile-time reification of concrete types.

2.5 Reflection and Runtime Type Information

The implementation of parametric polymorphism in Java is fundamentally based on *type erasure*. Unlike languages adopting a reified representation of generic types, the Java compiler translates every parameterized type into a non-generic representation before bytecode generation. During compilation, each type parameter is replaced by its upper bound if one is declared, or by `java.lang.Object` otherwise. In addition, the compiler automatically inserts the necessary type casts to preserve static type safety and generates bridge methods when required to maintain binary compatibility under inheritance and method overriding.

Consequently, generic type parameters exist exclusively within the static type system enforced by the compiler. The generated JVM bytecode contains no runtime distinction between different instantiations of the same generic declaration. For example, the declarations `List<String>` and `List<Integer>` are both translated into the same runtime representation corresponding to the raw type `java.util.List`. The Java Virtual Machine therefore executes a single runtime implementation regardless of the concrete type arguments that were present in the source program.

This compilation strategy implies that parameterized type arguments are not represented as independent runtime entities. The JVM preserves metadata describing the generic declaration itself through the class file's *Signature* attribute, allowing reflective APIs to reconstruct generic declarations under specific circumstances. However, the runtime representation of ordinary object

instances contains only the erased type, meaning that dynamically allocated objects do not retain the concrete type arguments with which they were instantiated.

2.5.1 Reflection and RTTI Limitations

Java's Runtime Type Information (RTTI) mechanism is centered on the `java.lang.Class` object associated with every loaded class. Since all parameterized instantiations of a generic class are erased to the same runtime type, reflection operates exclusively on the erased representation rather than on the original parameterized form.

A direct consequence is that parameterized generic types cannot participate in runtime type tests. The Java Language Specification explicitly prohibits expressions such as

```
obj instanceof List<String>
```

because the runtime system has no information capable of distinguishing `List<String>` from `List<Integer>`. Allowing such expressions would incorrectly suggest that the JVM could recover type arguments that no longer exist after compilation.

Similarly, the `getClass()` method always returns the runtime class corresponding to the erased type. Two objects instantiated with different generic arguments therefore produce identical runtime class descriptors.

```
import java.util.*;

public class ReflectionExample {

    public static void main(String[] args) {

        List<String> strings = new ArrayList<>();
        List<Integer> integers = new ArrayList<>();

        System.out.println(strings.getClass());
        System.out.println(integers.getClass());

        System.out.println(strings.getClass()
                               == integers.getClass());

        // Compilation error:
        // if (strings instanceof List<String>) { }
    }
}
```

The program produces equivalent runtime class objects for both lists, demonstrating that the generic arguments have been removed during compilation. The comparison evaluates to `true` because both variables reference instances of the same erased runtime class.

Reflection APIs such as `Class`, `Method`, and `Field` therefore identify only erased runtime types during ordinary object inspection. Although generic declarations remain partially accessible through reflective interfaces such as `java.lang.reflect.Type` and `ParameterizedType`, this information originates from generic signatures stored in class metadata rather than from the runtime representation of individual object instances. Consequently, reflection cannot determine the concrete type argument associated with an arbitrary generic object unless that information has been preserved explicitly by the programmer.

2.5.2 Type Tokens as an Explicit Runtime Representation

Because type erasure removes concrete generic arguments from ordinary runtime objects, Java libraries frequently employ *type tokens* to preserve type information explicitly. A type token consists of passing a `Class<T>` object alongside the generic value, thereby making the runtime type available independently of the erased generic parameter.

The following example illustrates this approach.

```
public class Box<T> {

    private final Class<T> type;
    private final T value;

    public Box(Class<T> type, T value) {
        this.type = type;
        this.value = value;
    }

    public Class<T> getType() {
        return type;
    }

    public T getValue() {
        return value;
    }

    public static void main(String[] args) {

        Box<String> box = new Box<>(String.class, "Hello");

        System.out.println(box.getType().getName());

        if (box.getType() == String.class) {
            System.out.println("String detected.");
        }
    }
}
```

The runtime type information in this example is not recovered from the generic parameter itself but from the explicitly supplied `Class<String>` object. This pattern is extensively employed by serialization frameworks, dependency injection containers, persistence libraries, and reflection-based infrastructures that require runtime knowledge of application types despite Java's erased implementation of generics.

2.5.3 Comparison with C++ Template RTTI

The reflection semantics of Java differ fundamentally from those of C++ because the two languages adopt distinct implementation strategies for parametric polymorphism.

C++ templates are implemented through *monomorphization*. During compilation, every template specialization generates an independent concrete type with its own layout, member functions, and associated runtime type metadata. Consequently, specializations such as `std::vector<int>` and `std::vector<double>` constitute distinct runtime types rather than different views of a shared implementation.

When Run-Time Type Information is enabled, the C++ runtime associates each polymorphic specialization with its own RTTI structures. Facilities such as `typeid` and `dynamic_cast` therefore operate on specialized concrete types generated by template instantiation. Each specialization possesses an independent `std::type_info` object corresponding to its unique runtime identity.

Java follows the opposite approach. All parameterized instantiations of a generic declaration are translated into a single erased runtime class represented by one shared `java.lang.Class` object. Reflection therefore exposes the identity of the generic declaration after erasure rather than the original parameterized instantiation appearing in the source code.

The distinction is summarized as follows:

- Java implements parametric polymorphism through type erasure, causing all parameterized instantiations of a generic class to share a single runtime type.

- Java reflection and RTTI operate exclusively on erased runtime classes, preventing direct runtime discrimination between different generic arguments.
- C++ implements templates through monomorphization, generating distinct concrete types for every specialization.
- C++ RTTI associates independent runtime metadata with each specialization, enabling facilities such as `typeid` and `dynamic_cast` to distinguish specialized template types directly.

These contrasting implementation strategies illustrate the fundamental trade-off between preserving runtime type identity through reification, as in C++, and reducing runtime type diversity through type erasure, as adopted by the Java Virtual Machine.

Chapter 3

C# Generics

3.1 Instantiation Time

C# generics adopt a runtime-oriented instantiation model that differs fundamentally from both the compile-time monomorphization strategy of C++ templates and the type erasure strategy of Java generics. The defining characteristic of the C# approach is that generic type definitions are compiled before execution, while the generation of their concrete executable representation is delayed until runtime by the Common Language Runtime (CLR) and its Just-In-Time (JIT) compiler.

In C#, generic declarations are not immediately expanded into specialized versions during compilation. Instead, the C# compiler translates generic classes and methods into Common Intermediate Language (CIL), preserving generic parameters as metadata entities. The resulting assembly contains a generic type definition together with information about its type parameters and constraints, but it does not necessarily contain native code for each possible generic instantiation.

For example, the generic definition:

```
class Container<T>
{
    public T value;
}
```

is compiled once into a generic CLI type definition. The compiler does not generate separate implementations such as:

```
Container<int>
Container<double>
Container<string>
```

at compilation time. The concrete instantiation is postponed until execution.

3.1.1 CLR and JIT Generic Instantiation Mechanism

The instantiation process begins when the CLR loads an assembly containing a generic definition. At this stage, the runtime knows the existence of the generic type but does not necessarily generate machine code for any concrete specialization.

A concrete generic instantiation is created when program execution requires a specific constructed type. For example, when the runtime encounters:

```
Container<int> c;
```

the CLR resolves the generic definition `Container<T>` together with the type argument `int`. This produces a runtime representation of the constructed generic type `Container<int>`.

Only when executable code for this instantiation is required does the JIT compiler translate the corresponding CIL into native machine instructions. Therefore, the timeline of C# generic

instantiation can be summarized as:

1. The C# compiler generates a generic CIL definition.
2. The assembly is loaded by the CLR.
3. A concrete generic instantiation is requested during execution.
4. The CLR creates the constructed generic type.
5. The JIT compiler generates native code when required.

Consequently, C# generics are instantiated at runtime rather than during source compilation.

The CLR does not necessarily create a completely independent native implementation for every generic instantiation. Instead, the JIT compiler applies different strategies depending on the category of the type argument.

For reference types, the CLR can usually share the same generated native code between different instantiations because all reference types have compatible managed reference representations. Therefore, instantiations such as:

```
List<string>
List<object>
List<MyClass>
```

may reuse a single JIT-generated implementation.

For value types, the situation is different. Value types have different memory layouts and sizes, meaning that a single shared implementation cannot always be used. Therefore, the CLR JIT compiler generates specialized native code for different value-type instantiations:

```
List<int>
List<double>
```

represent separate JIT specialization events.

Thus, the C# instantiation model is hybrid: runtime instantiation occurs in both cases, but reference types generally use runtime code sharing, whereas value types trigger runtime specialization.

3.1.2 Comparison with C++ Templates: Runtime Instantiation vs Compile-Time Instantiation

The C# runtime instantiation model differs from the C++ template model primarily because the moment at which specialization occurs is different.

C++ templates use compile-time instantiation, commonly described as monomorphization. When the compiler encounters:

```
Vector<int>
Vector<double>
```

it generates separate concrete implementations during compilation. The executable produced by the compiler already contains the specialized versions before execution.

The timeline is therefore:

Template Definition → Compile-Time Instantiation → Native Code Generation → Execution

In contrast, the C# timeline is:

Generic Definition → CIL Generation → Runtime Instantiation → JIT Compilation → Execution

The essential difference is that C++ performs specialization ahead of execution, whereas C# delays specialization until the runtime environment determines that a specific generic instantiation is required.

3.1.3 Comparison with Java Generics: Runtime Instantiation vs Type Erasure

Java generics follow a fundamentally different model based on type erasure. During Java compilation, generic parameters are removed and replaced by their erasures. Consequently, the Java Virtual Machine does not instantiate generic types at runtime.

For example:

```
List<String>
List<Integer>
```

are compiled into the same erased representation:

```
List
```

The JVM executes a single non-specialized implementation, and no runtime generic instantiation event occurs.

The timeline of Java generics is therefore:

Generic Source Code → Compile-Time Erasure → Bytecode Generation → Execution

This differs from C#, where the generic definition survives compilation and concrete instantiation is performed during execution.

Therefore, the three models can be summarized as:

Language	Instantiation Time	Specialization Phase
C++	Compile time	Compiler monomorphization
C#	Runtime	CLR/JIT instantiation
Java	Compile time	Type erasure, no specialization

The defining property of C# generics is therefore not merely the preservation of generic information, but the placement of the instantiation event: generic types are materialized by the runtime system when execution requires them, with the JIT compiler acting as the specialization mechanism.

3.2 Runtime Representation

C# generics provide a runtime implementation of parametric polymorphism based on *reification*. In a reified generic system, information about generic type parameters is preserved after compilation and remains available to the execution environment.

From a theoretical perspective, parametric polymorphism allows a program component to operate independently from the concrete types on which it is instantiated. A generic definition can be represented as:

$$G\langle T \rangle$$

where T is a type variable that can later be replaced by a concrete type argument. However, the concept of parametric polymorphism does not define how generic abstractions are represented at runtime. The implementation requires decisions about metadata representation, code generation, object layout, and execution mechanisms.

The CLR implements generics through a hybrid model that combines runtime reification with selective specialization. Generic definitions are preserved inside compiled assemblies, while the final executable representation is generated dynamically by the Just-In-Time (JIT) compiler.

Conceptually, the C# compilation pipeline can be represented as:

C# Source → CIL + Metadata → CLR Generic Instantiation → JIT Compilation → Native Code

This architecture allows the runtime to preserve generic type identity while deciding dynamically whether a particular instantiation requires a specialized implementation or whether an existing

implementation can be shared.

3.2.1 CLR Execution Engine and CIL Metadata Representation

The first stage of the CLR generic implementation occurs during compilation. The C# compiler translates source code into *Common Intermediate Language* (CIL) and generates metadata according to the Common Language Infrastructure (CLI) specification defined by ECMA-335.

Unlike Java bytecode after type erasure, the generated assembly preserves explicit information about generic declarations. A generic type is represented as an open generic definition containing formal type parameters.

For example:

$$\text{List}\langle T \rangle$$

is stored as a generic type definition rather than as a concrete implementation.

The CLR metadata system contains dedicated structures for representing generic relationships. The most relevant metadata tables include:

- **GenericParam**, which stores information about generic parameters;
- **GenericParamConstraint**, which stores constraints associated with generic parameters;
- **TypeSpec**, which represents constructed types;
- **MethodSpec**, which represents constructed generic methods.

These structures allow the CLR loader and execution engine to reconstruct the relationship between a generic definition and its concrete arguments.

For example, the runtime can represent:

$$\text{List}\langle T \rangle \rightarrow \text{List}\langle \text{int} \rangle$$

as an explicit construction relationship, preserved as part of the runtime type system rather than resolved away as in a type-erasure model.

3.2.2 Runtime Instantiation and JIT Compilation

A fundamental characteristic of CLR generics is that instantiation occurs at runtime rather than during the initial compilation phase. The compiler emits generic definitions, while the CLR creates concrete representations when a closed constructed type is required.

A generic definition:

$$G\langle T \rangle$$

becomes a runtime constructed type:

$$G\langle U \rangle$$

where U is a concrete type argument.

For example:

$$\text{Dictionary}\langle T\text{Key}, T\text{Value} \rangle$$

may generate runtime instantiations such as:

$$\text{Dictionary}\langle \text{int}, \text{string} \rangle$$

or:

`Dictionary<Guid, Object>`

The CLR execution engine resolves these constructed types and provides the necessary information to the JIT compiler. The JIT compiler then generates native machine code according to the characteristics of the supplied type arguments, performing this instantiation step as part of runtime execution rather than during compilation.

3.2.3 Value-Type Specialization

When a generic type is instantiated with a value type, the CLR generally produces a specialized native implementation for that concrete type. Value types such as `int`, `double`, or user-defined structures have their own memory layouts and representation requirements. Consequently, sharing a single machine-code implementation between unrelated value types would require additional abstraction mechanisms.

For example:

`List<int>`

and:

`List<double>`

require different representations of their stored elements. Therefore, the JIT compiler generates specialized code:

$JIT(List<int>) = NativeCode_{int}$

and:

$JIT(List<double>) = NativeCode_{double}$

The generated implementations contain concrete type information and operate directly on the corresponding value representation, produced by the CLR at runtime rather than by the compiler ahead of execution.

3.2.4 Reference-Type Code Sharing

Generic instantiations based on reference types follow a different execution strategy. Because all reference types share a common object-reference representation inside the CLR, the runtime can reuse the same native implementation for multiple generic instantiations.

For example:

`List<string>`

and:

`List<object>`

may share the same generated machine code.

Conceptually, instead of producing:

$JIT(List<string>) = NativeCode_{string}$

and:

$$JIT(List<object>) = NativeCode_{object}$$

the CLR can generate a shared implementation:

$$NativeCode_{reference}$$

where runtime-specific information is supplied through additional execution-engine structures.

This mechanism provides a balance between specialization and code reuse: the CLR avoids generating an independent implementation for every reference-type instantiation, while still retaining the exact generic type arguments needed to preserve type safety at runtime.

3.2.5 Runtime Dictionaries and Method Tables

To support shared generic implementations, the CLR uses runtime data structures that provide information about concrete generic arguments. These structures are commonly described as generic dictionaries or runtime generic contexts.

When shared machine code executes, operations requiring type-specific information can access these runtime structures to resolve:

- type descriptors;
- method addresses;
- field layouts;
- interface implementations.

This mechanism is conceptually related to dictionary passing techniques used in implementations of parametric polymorphism. The generated code remains generic, while additional runtime information supplies the missing concrete type details – a mechanism that has no equivalent in either C++ template instantiation, where all parameters are resolved before execution, or Java’s type-erasure model, where generic parameters are removed before bytecode execution rather than resolved at run time.

3.2.6 Runtime Type Identity

A defining property of CLR reification is that constructed generic types maintain their runtime identity. The CLR considers different generic instantiations as distinct runtime types.

Therefore:

$$RuntimeType(List<int>) \neq RuntimeType(List<string>)$$

This capability is fundamentally different from Java, where:

$$RuntimeType(List<String>) = RuntimeType(List<Integer>) = List$$

after erasure.

C++ occupies a different position. Concrete template specializations may participate in RTTI if they correspond to runtime types, but the original template definition is not preserved as a runtime entity. The executable contains information about concrete types rather than about the generic abstraction that produced them. The reflection system built on top of this runtime type identity is discussed in Section 3.5.

3.2.7 Architectural Comparison

The runtime representation strategies can be summarized as follows:

Feature	C# Generics	Java Generics	C++ Templates
Generic model	Reification	Type Erasure	Monomorphization
Metadata presence	Preserved in CLR	Removed from JVM execution	Not available
Instantiation time	Runtime/JIT	Compile-time only	Compile-time
Code generation	Specialization and sharing	Single erased byte-code	Native specialization
Runtime identity	Generic arguments preserved	Generic arguments removed	Concrete types only
Generic runtime object	Yes	No	No

Table 3.1: Comparison of generic runtime representation models.

3.3 Performance Characteristics

C# generics adopt a runtime reification model implemented by the .NET Common Language Runtime (CLR). Unlike Java Generics, where generic information is removed through type erasure, C# generic type parameters are preserved inside the Common Intermediate Language (CIL) metadata and remain available during execution. This allows the runtime environment to distinguish different generic instantiations and apply different execution strategies depending on the concrete type.

The CLR combines generic reification with Just-In-Time (JIT) compilation. Generic classes and methods are initially compiled into a type-independent CIL representation. When a concrete instantiation is required, the JIT compiler generates native machine code according to the characteristics of the involved type. Therefore, C# generics do not correspond to a purely compile-time expansion mechanism as in C++ templates, but they are also not completely erased as in Java.

The CLR execution strategy depends mainly on whether the generic parameter is a value type or a reference type:

$$\text{Generic}<T> \text{ execution} = \begin{cases} \text{specialized native code} & \text{if } T \text{ is a value type} \\ \text{shared native code} & \text{if } T \text{ is a reference type} \end{cases}$$

For value types such as `int`, `float`, or user-defined structures, the JIT compiler generates specialized implementations because different value types have different sizes, alignment requirements, and calling conventions. This allows direct manipulation of the actual data representation without additional indirection.

For reference types, instead, the CLR can share the same native implementation between different instantiations because object references have a uniform runtime representation. The execution engine maintains the required runtime type information through internal generic context structures, avoiding the need to generate duplicated machine code for every reference type.

This hybrid model provides a compromise between the complete specialization of C++ templates and the erased execution model of Java Generics.

3.3.1 Value-Type Instantiations: Memory Layout and Cache Efficiency

The main performance advantage of C# generics appears when generic data structures are instantiated with value types. In this case, the elements are stored directly inside the containing structure, avoiding object allocation and reference indirection.

Consider the following example:

```
public struct Vector3
{
```

```

    public float X;
    public float Y;
    public float Z;
}

public class Buffer<T>
{
    private T[] data;

    public Buffer(int size)
    {
        data = new T[size];
    }

    public float Compute()
    {
        float result = 0;

        for(int i = 0; i < data.Length; i++)
        {
            result += data[i].GetHashCode();
        }

        return result;
    }
}

```

When instantiated with a value type:

```

Buffer<Vector3> points =
    new Buffer<Vector3>(1000000);

```

the array contains the actual structures:

$$\boxed{(x_0, y_0, z_0) \mid (x_1, y_1, z_1) \mid (x_2, y_2, z_2)}$$

The memory representation is contiguous, allowing efficient sequential access. Because each element is stored directly inside the array, the processor can exploit spatial locality and efficiently use cache lines.

The execution cost is approximately:

$$T_{value} = N(C_{load} + C_{operation})$$

where the memory access component is minimized by predictable sequential loads.

Furthermore, no boxing operation is required. Primitive values and structures remain in their native representation, avoiding the additional memory accesses required by object wrappers. This makes C# generic collections of value types closer to the execution model of specialized C++ template instantiations.

3.3.2 Reference-Type Instantiations: Indirection and Shared Code

When a generic parameter is a reference type, the CLR uses code sharing. For example:

```

Buffer<VectorClass> objects =
    new Buffer<VectorClass>(1000000);

```

where:

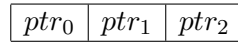
```

public class VectorClass
{
    public float X;
    public float Y;
}

```

```
public float Z;
}
```

the array stores references rather than the objects themselves:



The actual objects are located elsewhere in memory. Accessing an element requires an additional pointer dereference:

$$T_{reference} = N(C_{load\ reference} + C_{pointer\ chase} + C_{operation})$$

The additional memory access can reduce cache efficiency because the referenced objects may not be stored contiguously. Consequently, large collections of reference types generally exhibit weaker spatial locality compared with value-type collections.

However, unlike Java erased generics, this behavior does not require boxing of primitive values because C# maintains explicit runtime support for generic types.

3.3.3 Comparison with C++ Template Monomorphization

C++ templates use compile-time monomorphization. For every concrete template instantiation, the compiler generates a separate specialized implementation before program execution.

Example:

```
template<typename T>
class Buffer
{
    T* data;

public:
    T sum()
    {
        T result = 0;

        for(int i = 0; i < size; i++)
            result += data[i];

        return result;
    }
};

Buffer<float> a;
Buffer<double> b;
```

The compiler produces two independent versions:

$$Buffer < float > \rightarrow Code_{float}$$

$$Buffer < double > \rightarrow Code_{double}$$

This approach provides maximum optimization opportunities because the compiler has complete knowledge of the concrete type. The generated code can exploit static dispatch, aggressive inlining, and type-specific optimizations.

Compared with C#, C++ templates have the advantage that specialization happens before execution. However, every instantiation generates additional machine code, potentially increasing executable size and instruction-cache pressure.

C# generics avoid this duplication for reference types while still providing specialized execution for value types through JIT compilation.

3.3.4 Comparison with Java Type-Erased Generics

Java Generics follow a fundamentally different execution model based on type erasure. Generic parameters are removed during compilation and replaced with their upper bounds in the generated bytecode. A generic class:

```
class Buffer<T>
{
    private T[] data;

    T get(int index)
    {
        return data[index];
    }
}
```

is represented at runtime without knowledge of the original type parameter.

Because the JVM does not generate specialized versions for primitive types, generic collections require wrapper objects:

$$int \rightarrow Integer \rightarrow reference$$

For example:

```
ArrayList<Integer> values =
    new ArrayList<Integer>();
```

stores references to objects rather than compact primitive values. Accessing an element requires additional indirection and possible boxing/unboxing operations.

The resulting execution model introduces:

$$T_{Java} = T_{operation} + C_{boxing} + C_{indirection}$$

which can negatively affect memory locality and cache efficiency.

3.3.5 Summary of Runtime Performance Characteristics

The three approaches provide different trade-offs:

	C++ Templates	C# Generics	Java Generics
<i>Generic representation</i>	<i>Compile – timespecialization</i>	<i>Runtime reification</i>	<i>Type erasure</i>
<i>Value type storage</i>	<i>Direct</i>	<i>Direct</i>	<i>Usually boxed</i>
<i>Code generation</i>	<i>AOT monomorphization</i>	<i>JIT specialization</i>	<i>Shared erased code</i>
<i>Cache locality</i>	<i>Very high</i>	<i>High for value types</i>	<i>Lower due to indirection</i>
<i>Runtime overhead</i>	<i>Minimal</i>	<i>Low</i>	<i>Higher for primitives</i>

C# generics therefore represent an intermediate solution between C++ and Java. They preserve runtime type information and allow efficient specialized execution for value types, while avoiding excessive code duplication through reference type sharing. In terms of execution performance, C# approaches the efficiency of C++ templates for value-oriented workloads while providing a more flexible runtime model than Java erased generics.

3.4 Type Safety

Type safety represents one of the fundamental properties of a sound type system. A type-safe language guarantees that well-typed programs cannot perform operations that violate the rules

defined by the language type model. In the context of parametric polymorphism, type safety requires that generic abstractions preserve the relationships between type parameters and concrete types throughout the program.

From a theoretical perspective, a sound type system is generally described through the properties of *progress* and *preservation*. The preservation property states that the evaluation of a well-typed expression cannot invalidate its typing judgment:

$$\Gamma \vdash e : T \wedge e \rightarrow e' \implies \Gamma \vdash e' : T$$

where Γ represents the typing environment, e is an expression, and T is the type assigned by the type system.

The progress property states that a well-typed expression is either already a value or can perform a valid computation step:

$$\Gamma \vdash e : T \implies e \text{ is a value or } e \rightarrow e'$$

For generic programming, these principles imply that a generic abstraction must remain type-correct for every valid substitution of its type parameters. A generic definition cannot assume properties that are not guaranteed by the type system, and every operation performed on a type parameter must be justified by the available typing information.

C# generics implement parametric polymorphism while preserving these properties through static verification of generic type usage. The compiler verifies that all instantiations of a generic abstraction respect the constraints imposed by the generic declaration, preventing invalid substitutions from being accepted.

3.4.1 Static Type Checking of Generic Parameters

A generic type introduces a type variable whose concrete type is unknown during the declaration of the abstraction. However, this unknown type is not arbitrary: the type system reasons about it symbolically and ensures that every operation performed on the parameter is valid for all permitted substitutions.

Consider the following generic container:

```
public class Box<T>
{
    private T value;

    public void Set(T value)
    {
        this.value = value;
    }

    public T Get()
    {
        return value;
    }
}
```

The type parameter T represents an abstract type that will be replaced by a concrete type during generic instantiation. The type system guarantees that all values stored inside the container are compatible with the chosen instantiation.

For example:

```
Box<int> numbers = new Box<int>();

numbers.Set(10);
```

```
int value = numbers.Get();
```

The compiler verifies that the argument passed to `Set` is compatible with the type parameter substitution:

```
Box<int> numbers = new Box<int>();  
  
numbers.Set("text"); // Type error
```

The assignment is rejected because the generic instance `Box<int>` defines a type invariant requiring every stored value to be an integer. The generic mechanism therefore does not weaken static typing but extends it by allowing reusable abstractions while preserving exact type relationships.

3.4.2 Generic Constraints as Type System Rules

In a generic declaration, a type parameter does not automatically provide access to arbitrary operations. The type system only permits operations that are valid for every possible type satisfying the current assumptions about the parameter.

For example, the following generic method cannot assume that T provides a specific member:

```
public class Processor<T>  
{  
    public void Process(T value)  
    {  
        value.SpecialOperation();  
    }  
}
```

The compiler rejects this definition because the generic parameter does not guarantee the existence of `SpecialOperation`. The operation would violate type safety for substitutions where such a member does not exist.

Generic constraints extend the typing environment by introducing additional assumptions about the parameter:

```
public class Processor<T>  
    where T : IDisposable  
{  
    public void Process(T value)  
    {  
        value.Dispose();  
    }  
}
```

The constraint establishes that every valid substitution of T must implement `IDisposable`. Therefore, the invocation of `Dispose()` is statically verified as safe.

Constraints can be interpreted as additional typing rules restricting the set of types that can instantiate a generic abstraction. They do not represent runtime checks added after compilation, but rather static guarantees used by the type checker to prove correctness.

3.4.3 Variance and Type-Safe Generic Subtyping

Generic type safety also depends on the interaction between generics and subtyping. A naive interpretation of subtyping could incorrectly allow unsafe substitutions between parameterized types.

For example, although `string` is a subtype of `object`, a mutable generic container containing strings cannot generally be considered a container of objects:

```
List<string> strings = new List<string>();  
  
List<object> objects = strings; // Invalid
```

Allowing this assignment would violate type safety because an object of another type could be inserted into the list through the `List<object>` reference. The generic type system therefore treats mutable generic classes as invariant.

C# explicitly supports variance annotations for generic interfaces and delegates when the substitution is type-safe. For example:

```
IEnumerable<string> strings =  
    new List<string>();  
  
IEnumerable<object> objects = strings;
```

This assignment is valid because `IEnumerable<T>` only provides read-only access to elements. Since values are produced but never consumed through the generic interface, replacing a more specific type with a more general one cannot introduce an invalid operation.

Covariance and contravariance therefore represent controlled extensions of the subtyping relation, where the type system permits only substitutions that preserve soundness.

3.4.4 Reified Generics and Preservation of Type Information

C# generics are based on a reified model in which generic type parameters remain associated with the generic abstraction. From the perspective of type safety, this means that the relationship between a generic definition and its concrete type arguments is preserved.

A generic type declaration:

```
Box<T>
```

can be instantiated with different type arguments:

```
Box<int>  
Box<string>
```

These represent different constructed types derived from the same generic definition. The type system maintains the distinction between these parameterizations, ensuring that operations performed through one constructed type cannot violate the invariants of another.

This approach differs from type-erasure systems, where generic parameters are removed after compilation. In Java, for example, generic correctness is verified statically before erasure. The compiler guarantees that all uses of a generic abstraction are valid and may introduce implicit casts where necessary to preserve the source-level typing semantics.

Conceptually:

```
List<String> values = new ArrayList<>();  
  
String element = values.get(0);
```

is checked using the generic type information available during compilation, even though the executable representation does not preserve the original parameterized type.

C# follows a different type-system model: the association between the generic abstraction and its type parameter remains part of the program representation. Consequently, the generic type relationship is preserved rather than reconstructed after type erasure.

3.4.5 Comparison with C++ Templates from a Type System Perspective

C++ templates and C# generics both support parametric programming, but they represent different approaches to expressing type relationships.

In C++:

```
template<typename T>  
class Box  
{
```

```
T value;  
};
```

the compiler verifies whether a concrete substitution satisfies the requirements of the template when the template is instantiated. The resulting type correctness depends on the validity of the substituted operations.

C# generics instead define a parameterized type inside the language type system. The generic abstraction itself has a type-level meaning, and constraints explicitly describe the requirements that valid substitutions must satisfy.

Therefore, both mechanisms prevent invalid type usage, but they express generic correctness through different models:

- C++ templates rely on compile-time template substitution and validation of generated concrete types.
- C# generics represent parameterized types directly in the language type system and preserve the relationship between generic definitions and type arguments.

3.4.6 Type Safety Guarantees of the C# Generic Type System

The C# generic system preserves type safety through a combination of static checking rules and a precise representation of parameterized types. The main guarantees provided by the type system are:

- Every generic instantiation must satisfy the declared constraints of its type parameters.
- Operations performed on generic parameters are accepted only when they are valid for every permitted substitution.
- Generic subtyping relationships are restricted through variance rules to prevent unsafe substitutions.
- Generic abstractions preserve the association between type parameters and concrete types, maintaining the consistency of typing relationships.
- Invalid conversions between incompatible parameterized types are rejected statically.

Consequently, C# generics provide parametric polymorphism without sacrificing the guarantees of a statically typed language. The generic mechanism extends the expressiveness of the type system while preserving the fundamental property that well-typed programs cannot perform operations outside the set allowed by their declared types.

3.5 Reflection and Runtime Type Information

Unlike languages based on type erasure, the Common Language Runtime (CLR) implements a reified model of generics, where generic type information is preserved as part of the runtime representation of types. During compilation, C# source code is translated into Common Intermediate Language (CIL) instructions together with a rich metadata representation stored inside the assembly. Generic declarations, constraints, and instantiation relationships are encoded explicitly in this metadata and remain available during program execution.

A generic type declaration compiled into CIL is represented through metadata tables that preserve the association between a generic type definition and its parameters. For example, the declaration:

```
class Container<T>  
{  
}
```

is not transformed into a non-generic runtime entity. Instead, the CLR maintains a generic type definition with an explicit parameter placeholder. When a concrete instantiation such as:

```
Container<int>  
Container<string>
```

is requested, the runtime creates a closed constructed type whose metadata maintains the correspondence between the generic parameter and the concrete type argument.

This mechanism is commonly referred to as *generic reification*, because the abstract structure of the generic type remains represented in the runtime type system. Consequently, both open generic types, where type parameters are not yet bound, and closed constructed types, where every parameter has been substituted by a concrete runtime type, can be inspected through the reflection subsystem.

The CLR exposes this information through the `System.Type` abstraction. A runtime type object contains intrinsic identity information describing the represented entity, including whether the type is generic, whether it represents a generic definition or a constructed instance, and which runtime types are associated with its generic parameters.

For example, the following distinctions are maintained by the runtime:

- `typeof(List<>)` represents an open generic type definition.
- `typeof(List<int>)` represents a closed constructed generic type.
- `typeof(List<int>).GetGenericArguments()` returns the concrete runtime type argument associated with the instantiation.

Therefore, reflection over C# generics operates on actual runtime type entities rather than on erased compile-time abstractions.

3.5.1 Runtime Inspection of Generic Types Using `System.Reflection`

The following example demonstrates how the reflection API can dynamically inspect a generic object, verify whether its runtime type is generic, identify the generic type definition, and retrieve the concrete type arguments used for instantiation.

```
using System;
using System.Collections.Generic;

class ReflectionExample
{
    static void InspectType(object instance)
    {
        Type runtimeType = instance.GetType();

        Console.WriteLine("Runtime type: " + runtimeType);

        if (runtimeType.IsGenericType)
        {
            Console.WriteLine("The inspected type is generic.");

            Type genericDefinition =
                runtimeType.GetGenericTypeDefinition();

            Console.WriteLine(
                "Generic definition: " + genericDefinition);

            Type[] arguments =
                runtimeType.GetGenericArguments();

            Console.WriteLine("Generic arguments:");

            foreach (Type argument in arguments)
            {
                Console.WriteLine(argument.FullName);
            }
        }
        else
        {
            Console.WriteLine("The inspected type is not generic.");
        }
    }
}
```

```

    }
}

static void Main()
{
    List<int> values = new List<int>();

    InspectType(values);
}
}

```

In this example, the call to `GetType()` retrieves the CLR runtime type descriptor associated with the object instance. The method `IsGenericType` verifies whether the runtime entity corresponds to a generic type. If this condition is satisfied, `GetGenericTypeDefinition()` extracts the original generic declaration, while `GetGenericArguments()` retrieves the concrete runtime types used to construct the closed generic instance.

The result of the inspection is therefore equivalent to observing the following runtime relationship:

$$\text{List<int>} \rightarrow \text{List<>} + \text{int}$$

where both the generic declaration and the concrete type substitution remain available through reflection.

3.5.2 Comparison with C++ Templates and Java Generics

The reflection capabilities of C# generics differ fundamentally from the runtime models adopted by C++ templates and Java generics. The distinction concerns the availability of generic type information after compilation and the possibility of recovering concrete type parameters through runtime introspection.

C++ Templates: Compile-Time Monomorphization and Limited RTTI

C++ templates follow a compile-time monomorphization model. A template definition acts as a compile-time pattern from which the compiler generates specialized entities for each concrete instantiation. The relationship between a template parameter and its instantiated type is therefore resolved during compilation rather than represented as a generic runtime object.

For example:

```

template<typename T>
class Container
{
};

Container<int> a;
Container<double> b;

```

does not produce a runtime generic type equivalent to the CLR representation of `Container<T>`. Instead, the compiler materializes concrete types associated with the instantiated arguments. After compilation, the executable contains the resulting concrete classes but does not generally preserve a runtime representation of the original template abstraction.

The C++ Runtime Type Information (RTTI) mechanism provides limited type introspection through facilities such as `typeid` and `std::type_info`. RTTI can identify the dynamic type of polymorphic objects and expose implementation-defined type information, but it does not provide a reflection system capable of enumerating template parameters or reconstructing the template instantiation relationship.

Consequently, an expression such as:

```

typeid(Container<int>)

```

can identify the resulting concrete type, but standard C++ RTTI cannot query the template argument `int` as a runtime metadata element. The association between the template and its parameters exists primarily as a compile-time property.

Java Generics: Type Erasure and Residual Metadata

Java generics adopt a fundamentally different strategy based on type erasure. During compilation, generic type parameters are removed from the runtime representation and replaced according to erasure rules, typically by their upper bounds. The Java Virtual Machine therefore executes code where generic parameters are not represented as runtime type identities.

For example:

```
List<String> names = new ArrayList<>();
```

is compiled into a representation equivalent to a non-parameterized collection type from the perspective of runtime object identity. The JVM can identify that the object is an instance of `ArrayList`, but the concrete parameter `String` is not retained as part of the object's runtime type information.

Java reflection can inspect generic declarations when metadata has been preserved in class files through structures such as the `Signature` attribute. However, this information describes declarations rather than runtime object instantiations. For example, reflection can determine that a class was declared as:

```
class Box<T>
```

but it cannot directly recover the concrete argument used by an arbitrary runtime object:

```
Box<String>
```

because the instantiated parameter is not part of the runtime type identity.

Therefore, Java reflection operates on residual generic metadata associated with declarations, whereas C# reflection operates on reified runtime type entities.

3.5.3 Comparative Summary of Runtime Generic Reflection

Considering only runtime metadata availability and type introspection capabilities, the three approaches can be summarized as follows:

- **C# Generics:** generic parameters are reified by the CLR. Both open generic definitions and closed constructed types retain runtime metadata and can be inspected through `System.Reflection`.
- **C++ Templates:** template parameters are resolved through compile-time monomorphization. Standard RTTI identifies concrete runtime types but does not expose template parameter metadata.
- **Java Generics:** generic parameters are erased from runtime type identity. Reflection can access declaration-level generic information but cannot directly inspect concrete type arguments of runtime instances.

The CLR generic model therefore provides a reflection system in which generic type structure remains part of the runtime type universe, whereas C++ and Java represent opposite design points: compile-time expansion without generic runtime metadata in C++, and runtime abstraction through type erasure in Java.

Chapter 4

MetaProgramming

4.1 Overview of Metaprogramming

Metaprogramming is a programming technique in which a program can generate, analyze, or transform program entities rather than manipulating only runtime data. Its main objective is to move part of the computation to an earlier stage of the software lifecycle, such as compilation, or to enable dynamic modification and inspection of program structures during execution. The specific implementation of metaprogramming depends on the language compilation model and runtime architecture.

In **C++**, metaprogramming is primarily performed at compile time through template metaprogramming and constant expression evaluation mechanisms such as `constexpr`. Template instantiation allows the compiler to generate specialized program entities during translation, while `constexpr` enables the evaluation of deterministic expressions before runtime. Therefore, C++ metaprogramming is tightly integrated with the compiler and the static type system.

In **Java**, metaprogramming is mainly based on runtime mechanisms provided by the Java Virtual Machine (JVM), particularly reflection, annotations, dynamic proxies, and bytecode generation. Compile-time metaprogramming is more limited and is mainly supported through annotation processors. Due to type erasure, generic type parameters are generally not available in ordinary runtime objects, reducing the ability to perform type-based compile-time transformations.

In **C#**, metaprogramming combines compile-time and runtime approaches. Compile-time mechanisms include source generators, which produce additional code during compilation, while runtime mechanisms are provided by reflection, attributes, expression trees, and dynamic code generation facilities of the Common Language Runtime (CLR). Unlike Java, C# preserves generic type information at runtime through reified generics, enabling more extensive runtime type inspection.

Therefore, the main difference between these languages lies in when metaprogramming is performed: C++ emphasizes compile-time computation, Java primarily relies on runtime introspection, and C# provides a hybrid model combining compile-time generation with runtime analysis.

4.2 Compile-Time Computation Mechanisms in C++

C++ supports compile-time computation through language mechanisms that allow the compiler to evaluate expressions before runtime. The two primary mechanisms are recursive template instantiation and constant expression evaluation through `constexpr` functions. Although both approaches produce values known during compilation, they rely on different compiler rules: template metaprogramming performs computation through generation and substitution of template entities, whereas `constexpr` relies on compile-time evaluation of valid constant expressions.

The following examples assume a C++17 or later implementation.

4.2.1 Recursive Template Metaprogramming: Compile-Time Factorial

Traditional template metaprogramming performs computation by recursively instantiating templates. Each template specialization represents a distinct compile-time entity generated during semantic analysis.

```
#include <iostream>

template<unsigned N>
struct Factorial
{
    static constexpr unsigned value =
        N * Factorial<N - 1>::value;
};

template<>
struct Factorial<0>
{
    static constexpr unsigned value = 1;
};

int main()
{
    constexpr unsigned result = Factorial<6>::value;

    std::cout << result << '\n';

    return 0;
}
```

The expression:

```
Factorial<6>::value
```

requires the compiler to instantiate the corresponding template specializations. Template instantiation recursively generates the required compile-time entities:

$$\text{Factorial}<6> \rightarrow \text{Factorial}<5> \rightarrow \text{Factorial}<4> \rightarrow \dots \rightarrow \text{Factorial}<0>$$

The specialization:

```
template<>
struct Factorial<0>
```

terminates the recursive instantiation. During constant evaluation, the compiler substitutes the previously instantiated values:

$$\text{Factorial}(6) = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$$

The resulting value is available before code generation. The compiler does not emit instructions implementing recursive factorial evaluation. Instead, the instantiated expression is reduced to the constant value:

```
720
```

inside the compiler's intermediate representation. The template recursion exists only during compilation and is eliminated after semantic analysis. If subsequent optimization performs constant propagation, the generated assembly contains only the use of the already computed constant rather than runtime multiplication or recursive calls.

This behavior is guaranteed by the C++ template instantiation rules: all template arguments are known during translation, and template specialization resolution occurs before program execution.

Therefore, the computation is completed entirely before runtime.

4.2.2 Recursive Template Metaprogramming: Compile-Time Fibonacci

Template instantiation can also represent recursive computations with multiple dependencies. The following example evaluates the Fibonacci sequence during compilation.

```
#include <iostream>

template<unsigned N>
struct Fibonacci
{
    static constexpr unsigned value =
        Fibonacci<N - 1>::value +
        Fibonacci<N - 2>::value;
};

template<>
struct Fibonacci<0>
{
    static constexpr unsigned value = 0;
};

template<>
struct Fibonacci<1>
{
    static constexpr unsigned value = 1;
};

int main()
{
    constexpr unsigned result = Fibonacci<10>::value;

    std::cout << result << '\n';

    return 0;
}
```

When resolving:

Fibonacci<10>::value

the compiler recursively instantiates the required specializations until reaching the terminating cases:

Fibonacci<0>
Fibonacci<1>

The instantiated dependency structure represents the mathematical recurrence:

$$F(n) = F(n - 1) + F(n - 2)$$

After all required specializations have been generated, the compiler evaluates the constant expressions and obtains:

$$Fibonacci(10) = 55$$

The generated program does not contain the recursive Fibonacci computation. The template instantiation tree is a compile-time artifact used by the compiler to derive the final constant. After constant evaluation, references to the template member value are replaced by the computed result.

4.2.3 Compile-Time Evaluation Using constexpr Functions

Since C++11, the `constexpr` specifier allows functions to participate in constant expression evaluation. Unlike template metaprogramming, this mechanism does not generate specialized template entities. Instead, the compiler evaluates the function according to the rules defining valid constant expressions.

```
#include <iostream>

constexpr unsigned factorial(unsigned n)
{
    if (n == 0)
        return 1;

    return n * factorial(n - 1);
}

int main()
{
    constexpr unsigned result = factorial(6);

    std::cout << result << '\n';

    return 0;
}
```

Because `result` is declared as:

```
constexpr unsigned result
```

the initialization expression:

```
factorial(6)
```

must be evaluated during translation. The compiler invokes its constant expression evaluator, which interprets the function body according to the constant expression rules defined by the C++ standard.

The evaluation proceeds conceptually as:

$$\begin{aligned} factorial(6) &= 6 \times factorial(5) \\ &= 6 \times 5 \times factorial(4) \\ &= \dots \\ &= 720 \end{aligned}$$

After constant evaluation completes, the compiler substitutes the result into the program representation:

```
constexpr unsigned result = 720;
```

No runtime recursive execution is generated. The function body is used only by the compiler's constant evaluator during translation. The final executable contains only the operations required to use the computed value.

The guarantee that the computation occurs before runtime derives from the `constexpr` rules: an expression used to initialize a `constexpr` object must be a constant expression, meaning that its value must be determinable during compilation.

4.2.4 Compiler-Level Execution Model

Template metaprogramming and `constexpr` evaluation achieve compile-time computation through different compiler mechanisms.

Template-based computation is performed during semantic analysis through template instantiation, specialization matching, and substitution of template arguments. The compiler generates compile-time entities, evaluates their associated expressions, and replaces template references with the resulting values.

Conversely, `constexpr` computation is performed through constant expression evaluation. The compiler executes the function in an internal evaluation environment over its intermediate representation or abstract syntax tree, provided that every operation satisfies the constant expression requirements.

In both cases, the essential compilation transformation is:

compile-time expression $\rightarrow$ constant evaluation $\rightarrow$ generated program representation

The runtime program therefore begins execution only after the compile-time computation has already been completed.

4.2.5 Compile-Time Overload Resolution with SFINAE and C++20 Concepts

SFINAE (*Substitution Failure Is Not An Error*) enables compile-time selection of function overloads by controlling whether a template declaration participates in the overload resolution process. The following example uses `std::enable_if` to select different implementations according to compile-time properties of the template argument.

Listing 4.1: SFINAE-based overload selection using `std::enable_if`

```
#include <iostream>
#include <type_traits>

struct CustomType {};

template <typename T>
typename std::enable_if<std::is_integral<T>::value>::type
process(T value)
{
    std::cout << "Integral type: " << value << '\n';
}

template <typename T>
typename std::enable_if<std::is_floating_point<T>::value>::type
process(T value)
{
    std::cout << "Floating-point type: " << value << '\n';
}

template <typename T>
typename std::enable_if<
    !std::is_integral<T>::value &&
    !std::is_floating_point<T>::value
>::type
process(T)
{
    std::cout << "Custom or non-arithmetic type\n";
}

int main()
{
    process(42); // Integral overload
    process(3.14); // Floating-point overload
    process(CustomType{}); // Custom type overload
}
```

During template argument substitution, the compiler attempts to instantiate each candidate func-

tion by replacing the template parameter `T` with the deduced argument type. The expression contained in `std::enable_if` is evaluated during this substitution phase.

If the condition evaluates to `true`, `std::enable_if<T>::type` is defined and the resulting function declaration remains part of the overload resolution set. For example, for the call:

```
process(42);
```

the template parameter is deduced as `T = int`. The integral overload is transformed into:

```
typename std::enable_if<true>::type process(int);
```

which is a valid declaration because the nested type member exists.

Conversely, the floating-point overload undergoes the same substitution:

```
typename std::enable_if<false>::type process(int);
```

In this case, `std::enable_if<false>` does not provide a nested `type` member. The substitution therefore fails, causing this overload to be removed from the overload resolution set.

The failure is not considered a compilation error because SFINAE applies specifically during the substitution phase of template argument deduction. Invalid template substitutions eliminate the corresponding candidate instead of terminating compilation. A hard compilation error would occur only if the invalid expression appeared outside the immediate context of substitution or after overload resolution had already selected a candidate.

C++20 Concepts provide a more explicit mechanism for expressing the same compile-time constraints. Instead of relying on substitution failure, constraints are directly attached to template declarations and evaluated by the compiler during overload resolution.

Listing 4.2: C++20 Concepts-based overload selection

```
#include <iostream>
#include <concepts>

struct CustomType {};

template <typename T>
requires std::integral<T>
void process(T value)
{
    std::cout << "Integral type: " << value << '\n';
}

template <typename T>
requires std::floating_point<T>
void process(T value)
{
    std::cout << "Floating-point type: " << value << '\n';
}

template <typename T>
requires (!std::integral<T> && !std::floating_point<T>)
void process(T)
{
    std::cout << "Custom or non-arithmetic type\n";
}

int main()
{
    process(42); // Integral overload
    process(3.14); // Floating-point overload
    process(CustomType{}); // Custom type overload
}
```

With Concepts, the compiler evaluates the associated constraints before performing overload resolution. A function template whose constraints are not satisfied is discarded from the overload resolution set directly, providing the same compile-time specialization behavior as SFINAE while expressing the type requirements explicitly.

4.3 Compile-Time Overload Resolution with SFINAE and C++20 Concepts

SFINAE (*Substitution Failure Is Not An Error*) enables compile-time selection of function overloads by controlling whether a template declaration participates in the overload resolution process. The following example uses `std::enable_if` to select different implementations according to compile-time properties of the template argument.

Listing 4.3: SFINAE-based overload selection using `std::enable_if`

```
#include <iostream>
#include <type_traits>

struct CustomType {};

template <typename T>
typename std::enable_if<std::is_integral<T>::value>::type
process(T value)
{
    std::cout << "Integral type: " << value << '\n';
}

template <typename T>
typename std::enable_if<std::is_floating_point<T>::value>::type
process(T value)
{
    std::cout << "Floating-point type: " << value << '\n';
}

template <typename T>
typename std::enable_if<
    !std::is_integral<T>::value &&
    !std::is_floating_point<T>::value
>::type
process(T)
{
    std::cout << "Custom or non-arithmetic type\n";
}

int main()
{
    process(42); // Integral overload
    process(3.14); // Floating-point overload
    process(CustomType{}); // Custom type overload
}
```

During template argument substitution, the compiler attempts to instantiate each candidate function by replacing the template parameter `T` with the deduced argument type. The expression contained in `std::enable_if` is evaluated during this substitution phase.

If the condition evaluates to `true`, `std::enable_if<T>::type` is defined and the resulting function declaration remains part of the overload resolution set. For example, for the call:

```
process(42);
```

the template parameter is deduced as `T = int`. The integral overload is transformed into:

```
typename std::enable_if<true>::type process(int);
```

which is a valid declaration because the nested type member exists.

Conversely, the floating-point overload undergoes the same substitution:

```
typename std::enable_if<false>::type process(int);
```

In this case, `std::enable_if<false>` does not provide a nested `type` member. The substitution therefore fails, causing this overload to be removed from the overload resolution set.

The failure is not considered a compilation error because SFINAE applies specifically during the substitution phase of template argument deduction. Invalid template substitutions eliminate the corresponding candidate instead of terminating compilation. A hard compilation error would occur only if the invalid expression appeared outside the immediate context of substitution or after overload resolution had already selected a candidate.

C++20 Concepts provide a more explicit mechanism for expressing the same compile-time constraints. Instead of relying on substitution failure, constraints are directly attached to template declarations and evaluated by the compiler during overload resolution.

Listing 4.4: C++20 Concepts-based overload selection

```
#include <iostream>
#include <concepts>

struct CustomType {};

template <typename T>
requires std::integral<T>
void process(T value)
{
    std::cout << "Integral type: " << value << '\n';
}

template <typename T>
requires std::floating_point<T>
void process(T value)
{
    std::cout << "Floating-point type: " << value << '\n';
}

template <typename T>
requires (!std::integral<T> && !std::floating_point<T>)
void process(T)
{
    std::cout << "Custom or non-arithmetic type\n";
}

int main()
{
    process(42); // Integral overload
    process(3.14); // Floating-point overload
    process(CustomType{}); // Custom type overload
}
```

With Concepts, the compiler evaluates the associated constraints before performing overload resolution. A function template whose constraints are not satisfied is discarded from the overload resolution set directly, providing the same compile-time specialization behavior as SFINAE while expressing the type requirements explicitly.

4.4 Variadic C++ Template and other Metaprogramming Techniques

Variadic templates extend the C++ template system by allowing a template to accept an arbitrary number of type or non-type template arguments.

Introduced in C++11, they provide compile-time parameter pack abstraction, enabling the definition of generic components whose arity is not known a priori.

Traditional function or class templates require a fixed number of template parameters and cannot directly represent operations over heterogeneous collections of arbitrary size.

A typical application is recursive compile-time expansion of parameter packs:

```
#include <iostream>

template <typename T>
void print(T value) {
    std::cout << value << '\n';
}

template <typename T, typename... Ts>
void print(T first, Ts... rest) {
    std::cout << first << ' ';
    print(rest...);
}

int main() {
    print(1, 2.5, "text");
}
```

The template parameter pack `Ts...` represents an arbitrary sequence of types.

During template instantiation, the compiler generates specialized versions for each concrete argument list and recursively expands the pack until the base case is reached.

The mechanism is completely resolved at compile time through template instantiation and overload resolution.

4.4.1 Template Template Parameters

Template template parameters allow a template to receive another template as an argument.

They enable higher-order generic programming, where a class template operates on a family of compatible class templates rather than on individual instantiated types.

Ordinary templates cannot express abstractions over template structures themselves because their parameters represent types rather than templates.

A generic container adapter can be implemented as follows:

```
#include <vector>
#include <list>

template <typename T, template <typename> class Container>
class Stack {

    Container<T> data;

public:

    void push(const T& value) {
        data.push_back(value);
    }

    T pop() {
        T value = data.back();
        data.pop_back();
        return value;
    }
};

int main() {
```

```

    Stack<int, std::vector> a;
    Stack<int, std::list> b;
}

```

The parameter `Container` represents a template accepting a type parameter.

During compilation, the compiler substitutes the provided template argument and instantiates the resulting specialization.

This mechanism enables abstraction over template families while preserving compile-time type checking.

4.4.2 Curiously Recurring Template Pattern (CRTP)

The Curiously Recurring Template Pattern (CRTP) is a template-based inheritance technique in which a derived class passes itself as a template argument to its base class.

It enables static polymorphism by allowing the base class to invoke derived-class functionality through compile-time binding instead of runtime dynamic dispatch.

A minimal CRTP implementation is:

```

#include <iostream>

template <typename Derived>
class Interface {

public:

    void execute() {
        static_cast<Derived*>(this)->implementation();
    }
};

class Concrete : public Interface<Concrete> {

public:

    void implementation() {
        std::cout << "Concrete implementation";
    }
};

int main() {

    Concrete obj;
    obj.execute();
}

```

The base class template is instantiated with the derived type, producing a specialized class where calls to derived operations are resolved during compilation.

Unlike ordinary class templates, CRTP addresses the problem of expressing polymorphic relationships between types without requiring a runtime interface.

The mechanism relies on template instantiation and static dispatch, with the compiler generating a concrete implementation for each derived type participating in the pattern.

4.5 Limits of Java Generics in Replicating C++ SFINAE and Type Traits

Java generics provide bounded polymorphism through explicit type constraints, allowing the compiler to restrict the set of admissible type arguments.

A generic parameter can specify an upper bound, enabling compile-time verification of operations supported by the bound:

```
public class Calculator<T extends Number> {  
    public double square(T value) {  
        return value.doubleValue() * value.doubleValue();  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Calculator<Integer> c1 = new Calculator<>();  
        Calculator<Double> c2 = new Calculator<>();  
    }  
}
```

The constraint `T extends Number` performs compile-time bounds checking: only types satisfying the declared subtype relationship can instantiate the generic class.

However, Java generic bounds express only admissible type relationships and the operations provided by the bound; they cannot introduce conditional compile-time branching based on the concrete identity of `T`.

Unlike C++ templates, Java generics do not support SFINAE-style overload selection because generic instantiation does not generate independent specialized versions of generic code for each concrete type.

In C++, template substitution and type-traits evaluation occur during compilation, allowing the compiler to select or discard overloads depending on whether expressions involving a type are valid.

This behavior relies on compile-time type substitution and template monomorphization.

Java generics are instead implemented through type erasure.

After compilation, generic type parameters are removed from the generated bytecode, and generic classes are represented using their erased upper bounds.

Consequently, a generic method cannot perform compile-time branching based on properties that are specific to an unknown type parameter:

```
public static <T> void process(T value) {  
    // Impossible:  
    // select implementation depending on properties of T  
}
```

The compiler cannot instantiate separate versions of `process` for different values of `T`, nor can it use the concrete type argument to enable or disable overloads during compilation.

Any type-dependent behavior must therefore rely on mechanisms available after erasure, such as explicit runtime type information, rather than compile-time substitution analysis.

Therefore, Java bounded polymorphism provides static constraints on generic parameters but cannot reproduce the compile-time introspection and conditional specialization capabilities of C++ SFINAE and type traits.

4.6 Limits of C# Generics in Replicating C++ SFINAE and Type Traits

C# does not provide a direct equivalent of C++ SFINAE or compile-time type traits, since generic constraints are based on bounded polymorphism rather than substitution-based overload resolution.

However, the combination of generic constraints introduced by the ‘where’ clause and static abstract interface members introduced in .NET 7 allows C# to express a restricted form of trait-oriented generic programming.

A classical C++ type-traits pattern uses SFINAE to enable an overload only when a type satisfies a compile-time property:

```
template<typename T>
auto add(T a, T b) ->
    std::enable_if_t<std::is_arithmetic_v<T>, T>
{
    return a + b;
}
```

During template argument substitution, the compiler attempts to instantiate the candidate function.

If the required type property is not satisfied, the substitution failure removes the overload from the candidate set without generating a compilation error.

The closest C# abstraction is obtained through constrained generic parameters:

```
static T Add<T>(T a, T b)
    where T : INumber<T>
{
    return a + b;
}
```

The constraint ‘where T : INumber<T>’ restricts the admissible type arguments to types implementing the required numeric contract.

The compiler validates the generic instantiation against the declared bounded polymorphic constraint, ensuring that all operations required by the generic algorithm are available for the selected type.

The interface-based approach enabled by static abstract interface members provides a trait-like mechanism:

```
public interface IAddable<TSelf>
    where TSelf : IAddable<TSelf>
{
    static abstract TSelf operator +(TSelf x, TSelf y);
}
```

A generic algorithm can therefore depend on operations defined by a type-level contract.

The compiler checks the presence of the required static members during generic type validation, providing functionality conceptually similar to C++ concepts and type traits, although with a more restricted expressive model.

This capability allows C# to approximate C++ trait-based programming more closely than Java generics.

The reason is that .NET generics use runtime reification: generic type arguments and constraints are preserved in metadata and remain available to the CLR type system.

Therefore, the compiler and runtime can reason about the actual generic arguments and their implemented interfaces.

Java generics, based on type erasure, cannot provide equivalent guarantees over generic type capabilities.

Nevertheless, C# generic constraints cannot implement true SFINAE.

Constraint violations are compilation errors rather than substitution failures that selectively remove invalid candidates during overload resolution.

The C# type system does not provide arbitrary compile-time reflection over type structure, conditional declaration generation, or unrestricted compile-time computation.

Generic constraints express statically known relationships between types, but they do not form a general compile-time metaprogramming mechanism.

This limitation follows from the different implementation strategies of the two languages.

C++ templates rely on compile-time substitution and monomorphization, where templates are instantiated into concrete program fragments after AST-level substitution.

This enables arbitrary type-dependent specialization and compile-time computation.

C# generics instead rely on runtime reification, where generic definitions remain associated with CLR metadata and are validated according to declared constraints.

Consequently, C# generic constraints and generic math provide trait-like abstractions, but they cannot reproduce the substitution-based metaprogramming model of C++ templates.

4.7 Instantiation-Time Checking and Metaprogramming Expressiveness

C++ templates perform semantic checking after template argument substitution, during the instantiation phase. The template definition may contain dependent expressions whose validity cannot be determined until a concrete type is provided. This delayed checking enables compile-time selection mechanisms in which the existence or absence of operations on a type influences overload resolution. Java and C# generics adopt a different model: generic declarations must be type-correct independently of future instantiations, and only operations guaranteed by the declared bounds are permitted. Consequently, generic code cannot express arbitrary structural requirements that are discovered only when a type is substituted.

A representative C++ technique enabled by instantiation-time checking is SFINAE (Substitution Failure Is Not An Error). During overload resolution, if substituting a template argument makes a dependent expression invalid, the candidate is removed rather than producing a compilation error:

```
#include <type_traits>

template <typename T>
auto process(T value) -> decltype(value.serialize(), void())
{
    value.serialize();
}

template <typename T>
void process(T)
{
    // fallback implementation
}
```

For a type providing a `serialize()` member function, substitution succeeds and the first overload is selected. For a type without such a member, the expression `value.serialize()` becomes invalid during substitution; the failure is discarded by SFINAE and the fallback overload remains viable. The compiler therefore performs a form of structural capability detection without requiring an explicit interface or inheritance relationship.

The same mechanism cannot be directly expressed in Java or C# generics because generic declarations are checked before concrete type arguments are substituted. A generic method cannot contain an operation whose validity depends on members not guaranteed by the declared type bound:

```
static <T> void process(T value) {  
    value.serialize(); // compilation error  
}
```

The declaration is rejected because the type parameter `T` does not provide any guarantee that a `serialize()` operation exists. The compiler cannot defer this validity check until a future instantiation with a specific type. The only available mechanism is to explicitly encode the requirement through bounded polymorphism:

```
interface ISerializable {  
    void Serialize();  
}  
  
static void Process<T>(T value) where T : ISerializable  
{  
    value.Serialize();  
}
```

However, this constraint must be declared before generic instantiation and cannot be inferred from the structural properties of an arbitrary type. Unlike C++ template substitution, bounded polymorphism in Java and C# cannot perform compile-time capability detection based on the presence of members. The limitation follows directly from declaration-time checking: generic bodies must already be valid for all admissible type arguments, whereas C++ allows validity to be determined only after substitution during template instantiation.

4.8 External Metaprogramming Workarounds in Java and C#

Unlike C++ templates, whose instantiation mechanism is intrinsically integrated into the language type system and compiler pipeline, Java and C# generics do not provide native facilities for arbitrary compile-time program generation or type-directed code transformation.

Their generic systems primarily express parametric polymorphism through constrained type abstraction, while metaprogramming capabilities are provided through additional mechanisms operating outside the generic type system.

4.8.1 Java Reflection, Annotation Processing, and Code Generation

Java commonly relies on runtime reflection as a mechanism to recover and manipulate structural information that is unavailable through ordinary generic operations.

Reflection operates after compilation, using the metadata stored in class files to inspect classes, methods, fields, annotations, and runtime types.

Although it enables dynamic behavior based on type information, it does not participate in generic instantiation or compiler-driven code specialization.

The reflective model therefore represents runtime introspection rather than compile-time metaprogramming.

Compile-time metaprogramming in Java is instead commonly implemented through annotation processors based on the Java Annotation Processing API.

These processors execute during compilation and analyze annotated source elements through compiler-provided abstract representations of the program, such as the language model API.

They can generate additional source files, modify compilation workflows, or produce auxiliary classes, but they do not extend the semantics of Java generics.

The generated artifacts are independent program units produced before compilation rather than

specialized instances created by the generic type system.

Code generation libraries such as Lombok extend this approach by performing source-level transformations during compilation.

Lombok modifies compiler representations, commonly through abstract syntax tree manipulation, to inject methods, constructors, or other structural elements that developers would otherwise write manually.

However, these transformations are external compiler extensions and are not part of Java's generic substitution mechanism.

The generic system remains responsible only for type abstraction and checking, while the generation framework separately alters program structure.

Consequently, Java metaprogramming workarounds operate either dynamically through reflection or externally to the compiler type system through annotation-driven generation.

They do not provide a mechanism equivalent to C++ template instantiation, where template definitions themselves are evaluated by the compiler as parameterized program fragments.

4.8.2 C# Source Generators and T4 Templates

C# provides more integrated compile-time generation mechanisms than Java, but these remain architecturally separate from the generic type system.

Source generators, introduced with the Roslyn compiler platform, execute during compilation and analyze the program's syntax trees and semantic models.

They can emit additional C# source code that becomes part of the compilation unit, allowing automatic generation of serialization code, boilerplate implementations, or type-specific helpers.

Despite operating within the compilation pipeline, source generators do not participate in generic type instantiation.

A generic declaration is still processed according to the CLR generic model, while generated source code represents additional compilation input produced by an external transformation phase.

The generator does not extend generic constraints into a general compile-time evaluation mechanism and cannot directly replace the template instantiation model of C++.

Another common mechanism is the use of T4 (Text Template Transformation Toolkit) templates, which perform code generation through template files processed before compilation.

T4 templates are independent generation programs that produce source files according to explicit transformation rules.

Their execution resembles a preprocessing stage rather than a semantic phase of the C# type system, since the generated code is compiled afterward as ordinary C#.

The fundamental distinction is that Java and C# generics describe reusable abstractions within the language type system, whereas reflection, annotation processors, source generators, and T4 templates are auxiliary transformation mechanisms layered around the compiler or runtime environment.

C++ templates combine parametric polymorphism and compile-time program generation in a single language construct: template parameters directly control compiler evaluation, substitution, and specialization.

In Java and C#, equivalent behavior requires separate metaprogramming infrastructures that generate or inspect code outside the native generic abstraction mechanisms.

4.9 Summary Practical Limits of Metaprogramming in C++, Java, and C# Generics

Capability	C++ Templates	Java Generics	C# Generics
Compile-time computation	Supported — Template metaprogramming and <code>constexpr</code> evaluation allow computations to be performed during compilation through template instantiation and compile-time evaluation.	Not supported — Generic instantiation does not execute compile-time computations because Java generics are implemented through type erasure and are absent from runtime specialization.	Partially supported — Generic constraints and compile-time type checking provide limited static reasoning, but C# does not offer general compile-time computation over generic types.
Conditional compilation based on type properties	Supported — SFINAE, type traits, and Concepts enable compile-time selection of implementations according to properties of template arguments.	Not supported — Type erasure prevents compile-time branching on concrete type properties because generic parameters do not retain specialization information.	Partially supported — Generic constraints allow selection based on declared type relationships, but they cannot express arbitrary compile-time predicates over types.
Code generation per type	Supported — Template monomorphization generates separate specialized code for each instantiated type during compilation.	Not supported — Type erasure produces shared generic implementations that operate on erased types rather than generating specialized code per type argument.	Supported for value types and limited for reference types — CLR reification enables JIT-generated specialized implementations for value types while reference types generally share generic code.
Runtime type introspection	Partially supported — RTTI provides runtime information about concrete objects, but template parameters are not generally preserved as runtime metadata after compilation.	Partially supported — Reified class objects exist for some generic information, but type erasure removes most generic type arguments from runtime representation.	Supported — CLR reification preserves generic type information in metadata, allowing runtime inspection of instantiated generic types.

Table 4.1: Practical limits of metaprogramming capabilities in C++, Java, and C# generics.